影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 10 章 二值影像處理

本章學習目標

- 說明二值影像的意義與來源，理解型態學處理建立在集合論與結構元素之上。
- 推導並實作侵蝕、膨脹兩個基本運算，以及斷開、閉合兩個組合運算。
- 運用型態學梯度、頂帽與黑帽轉換解決邊緣萃取與不均勻背景校正問題。
- 操作 Hit-or-Miss、細線化、骨架化與孔洞填補等進階型態學技術。
- 以連通元件標記完成物件計數，並計算距離轉換供分割與量測使用。
- 理解 Haar 級聯分類器如何以積分影像與級聯結構達成即時人臉偵測，並在 ESP32-CAM 上部署。

叫獸開講|2001 年，相機學會了找人臉：Viola-Jones 的百萬分之一秒

在深度學習席捲電腦視覺的十一年前，有一個演算法就已經讓相機學會即時「看見」人臉了。2001 年，微軟研究院的 Paul Viola 與 Michael Jones 在頂級會議 CVPR 上發表了一篇論文《Rapid Object Detection using a Boosted Cascade of Simple Features》，提出了史上第一個能在普通電腦上即時執行的人臉偵測器。在當年一台 700MHz 的 Pentium III 處理器上，它的速度比同等準確度的其他方法快上數十甚至數百倍——這在當時是不可思議的成就。

Viola-Jones 的成功，靠的是三個環環相扣的巧思。第一是 Haar-like 特徵：它不看單一像素，而是計算「相鄰的矩形區域之間的亮度差」——例如，人臉上「眼睛區域通常比臉頰暗」，這個簡單的明暗關係就是一個特徵。第二是積分影像(Integral Image)：一張影像預先算好「每個點左上方所有像素的累加和」之後，任何一個矩形區域的像素總和，都只需要查表做三次加減法就能得到，與矩形大

小完全無關——這讓成千上萬個 Haar 特徵能以驚人的速度計算。第三是級聯分類器(Cascade)：把分類器串成一連串關卡，前面的關卡用極少的特徵、極快的速度，把「明顯不是臉」的區域(佔畫面絕大部分)迅速淘汰;只有通過前關的少數候選，才交給後面更精密(也更耗時)的關卡仔細檢查。這個「快速排除、集中火力」的設計，是它能即時執行的關鍵。

把這三招放在本章的脈絡下看，你會發現它們與二值影像處理的精神一脈相承。Haar 特徵計算的是矩形區域的加總與相減，本質上就是對區域做集合式的運算;積分影像是一種「預先累加」的加速技巧，和本章連通元件標記中「一次掃描就完成統計」的思路異曲同工;而級聯「逐步淘汰、由粗到精」的策略，更是型態學處理中「先粗略清理、再精細處理」的工程智慧。Viola-Jones 內建於 OpenCV 多年(即著名的 Haar Cascade)，至今仍是資源受限裝置(如本章要部署的 ESP32-CAM)上人臉偵測的實用選擇。

當然，它也有明顯的侷限：它最擅長偵測正面、端正的人臉，遇到側臉、大角度傾斜或嚴重遮擋就力有未逮;這些正是第 16 章深度學習物件偵測器要克服的。但 Viola-Jones 的歷史地位無可撼動——它證明了「精心設計的簡單特徵 + 聰明的加速結構 + 機器學習的特徵篩選」可以創造奇蹟，這個信念直到今天仍深深影響著整個領域。

思考題：級聯分類器「用快速的關卡先淘汰明顯不對的候選，把運算集中在少數可能的目標上」——這個「由粗到精、快速排除」的思想，在生活中還有哪些應用?(想想：履歷篩選、垃圾郵件過濾、機場安檢)

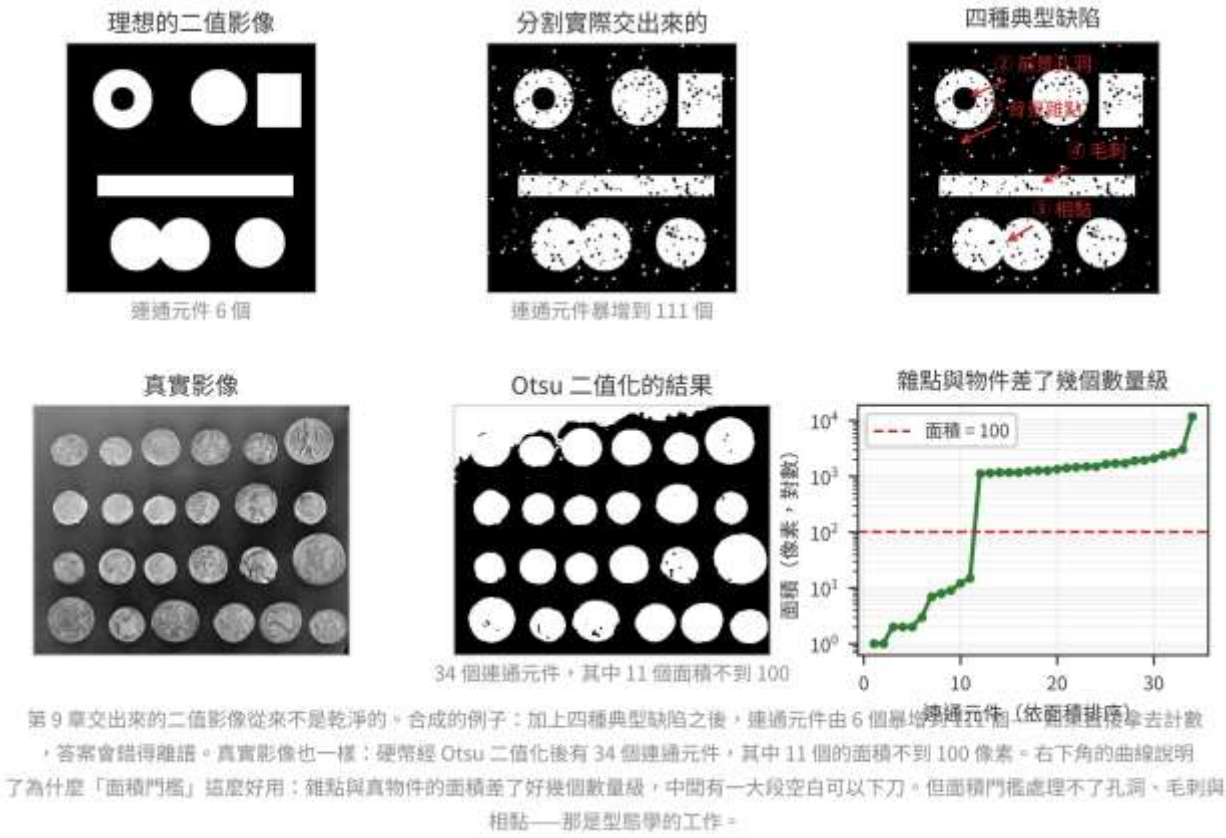
10.1 二值影像與型態學的基礎

10.1.1 什麼是二值影像

二值影像(Binary Image)是每個像素只有兩種可能值的影像：前景(通常設為 1 或 255，顯示為白)與背景(通常設為 0，顯示為黑)。它是第 9 章分割的直接產物——門檻

法、Otsu、邊緣偵測的輸出，幾乎都是二值影像。二值影像的資訊量雖然遠低於灰階或彩色影像，卻正是它的價值所在：當我們只關心「物件在哪裡、形狀如何、有幾個」時，把影像化約成非黑即白，能讓後續的形狀分析變得極為高效。

但分割產生的二值影像往往並不完美：前景區域中可能有小孔洞(雜訊或反光造成)、背景中散布著孤立的小白點(雜訊)、本該分開的物件黏在一起、本該相連的物件斷成幾截。本章的型態學運算，就是清理與修整這些缺陷、並從中萃取形狀資訊的工具箱——它是第 9 章分割與第 13 章特徵擷取之間承先啟後的關鍵環節。



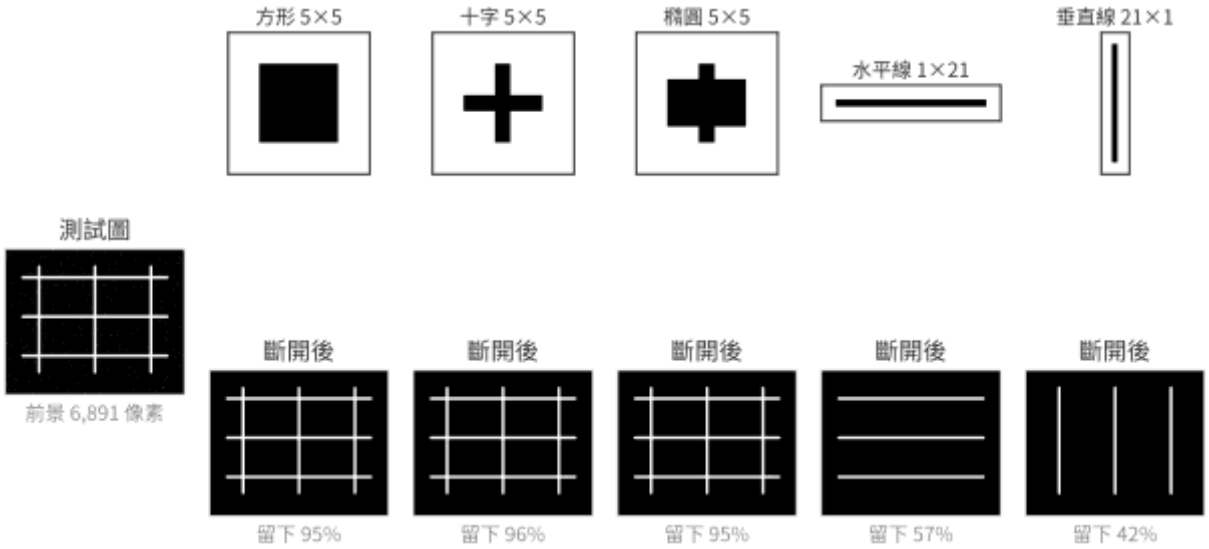
第 9 章交出來的二值影像從來不是乾淨的。合成的例子：加上四種典型缺陷之後，連通元件由 6 個暴增到 111 個，若直接拿去計數，答案會錯得離譜。真實影像也一樣：硬幣經 Otsu 二值化後有 34 個連通元件，其中 11 個的面積不到 100 像素。右下角的曲線說明了為什麼「面積門檻」這麼好用：雜點與真物件的面積差了好幾個數量級，中間有一大段空白可以下刀。但面積門檻處理不了孔洞、毛刺與相黏——那是型態學的工作。

圖 10-1　分割交出來的二值影像長什麼樣：四種典型缺陷

10.1.2 型態學與結構元素

數學型態學(Mathematical Morphology)由 Georges Matheron 與 Jean Serra 在 1960 年代創立於法國，原本用於分析礦石與地質切片的形狀，其理論基礎是集合論。核心觀念是：把二值影像的前景視為一個點集合，用一個稱為結構元素(Structuring Element)的小形狀去「探測」這個集合，依據結構元素與前景的貼合關係，決定輸出。

結構元素相當於空間濾波的「核」，但作用方式是集合運算而非數值運算。它可以是任意形狀——方形、十字形、圓形、線形——形狀的選擇會直接影響處理結果：用水平線形結構元素，會強化水平結構、削弱垂直結構;用圓形結構元素，則各方向一視同仁。結構元素還有一個「原點」(通常在中心)，決定它在影像上對齊的方式。選對結構元素的形狀與大小，是型態學處理的關鍵藝術。



同一張圖、同一個運算（斷開），只換結構元素的形狀，結果天差地別。方形、十字、橢圓這三個小的結構元素都只是去雜點，前景分別留下 95%、96%、95%;而 1×21 的水平線只留下水平線條（57%），21×1 的垂直線只留下垂直線條（42%）。這就是型態學與空間濾波最大的差別：濾波的核決定「怎麼加權」，結構元素決定「要找什麼形狀」。表格線萃取、PCB 線路分離、條碼定位，用的都是這一招。

圖 10-2　結構元素的形狀決定你探測到什麼

程式 10-1 建立結構元素

```
import cv2, numpy as np

# 建立不同形狀的結構元素
rect  = cv2.getStructuringElement(cv2.MORPH_RECT,    (5, 5))  # 方形
cross = cv2.getStructuringElement(cv2.MORPH_CROSS,   (5, 5))  # 十字
ellip = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))  # 橢圓

# 也可手動指定形狀,例如水平線(專門處理水平結構)
hline = np.zeros((1, 15), np.uint8); hline[:] = 1

print(cross)   # 觀察十字結構元素的形狀
```

10.2 兩個基本運算：侵蝕與膨脹

10.2.1 侵蝕

侵蝕(Erosion)的規則是：把結構元素滑過影像，只有當結構元素完全被包含在前景之內時，其原點對應的位置才保留為前景，否則設為背景。直觀的效果是：前景物件會「縮水」、邊界向內收縮，小於結構元素的細節(細小突起、細線、孤立雜點)會被完全消滅。

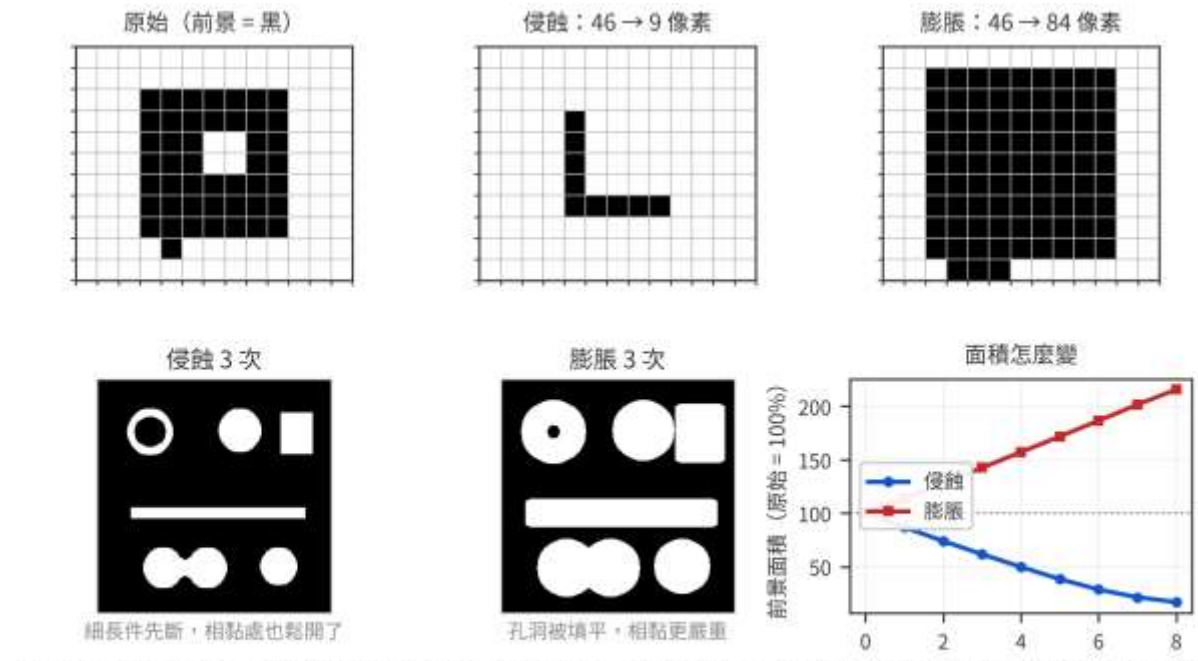


圖 10-3　侵蝕與膨脹：從一個小矩陣看清楚定義

侵蝕的典型用途：去除背景中的孤立雜點、斷開兩個以細頸相連的物件、縮小過度膨脹的前景。一個重要觀察(呼應第 7 章)：對灰階影像而言，侵蝕等同於在鄰域取最小值——這正是第 7 章提過的最小值濾波。同一個運算，在第 7 章是「去除鹽雜訊」，在這裡是「腐蝕形狀」，再次印證影像處理概念的一體多面。

10.2.2 膨脹

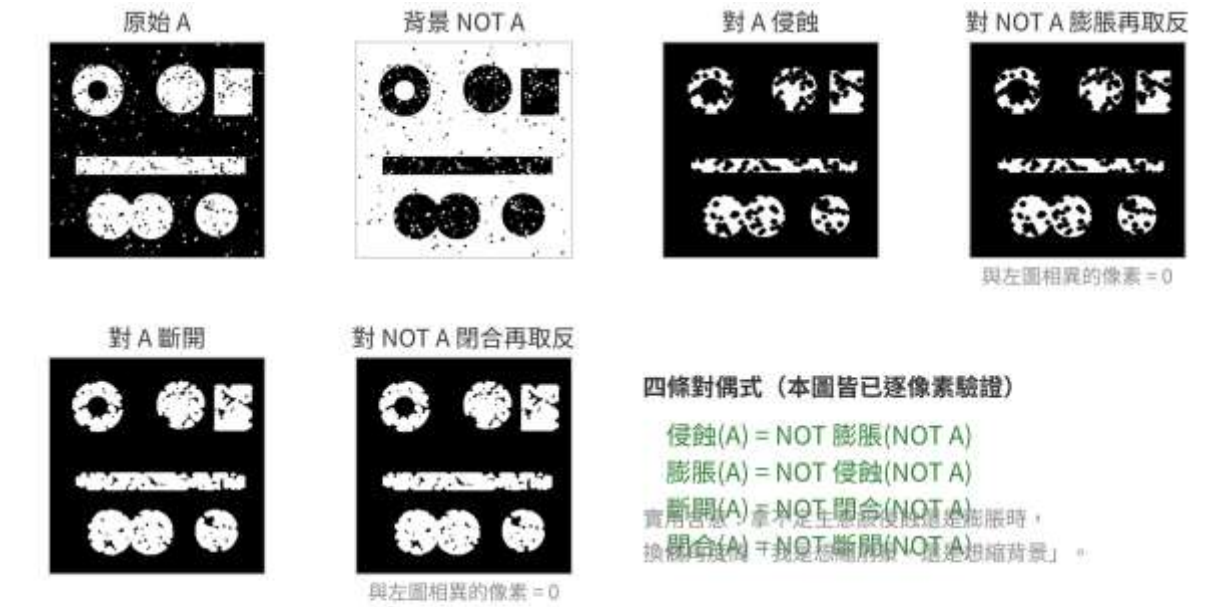
膨脹(Dilation)是侵蝕的對偶運算，規則是：只要結構元素與前景有任何交集，其原點對應的位置就設為前景。效果與侵蝕相反：前景物件會「長大」、邊界向外擴張，小的孔洞與裂縫會被填補、斷裂的部分可能重新相連。



圖 10-4　侵蝕就是最小值濾波，膨脹就是最大值濾波——但別只做一半

膨脹的典型用途：填補前景內的小孔洞、連接斷裂的線段或字元、放大前景以便後續處理。同樣呼應第 7 章：對灰階影像而言，膨脹等同於在鄰域取最大值，即最大值濾波(去除胡椒雜訊)。

侵蝕與膨脹是一對對偶運算：對前景侵蝕，等於對背景膨脹。理解這個對偶性，能幫助你在「該侵蝕還是該膨脹」之間快速做出判斷——問自己是想縮前景還是縮背景即可。



對偶性是型態學最省力的一條性質：對前景做的每一個運算，都等於對背景做對應的運算再取補集。本圖把兩條式子逐像素比對：「對 A 侵蝕」與「對背景膨脹再取反」相異的像素是 0 個；「對 A 斷開」與「對背景閉合再取反」相異的像素是 0 個。這表示你永遠只需要記住其中一半。實務上更重要的是它提醒你：前景與背景是對稱的，「哪一邊是前景」只是你自己的約定——若把二值影像反過來看（前景當背景），斷開與閉合的角色就互換了。

圖 10-5　對偶性：對前景侵蝕，就是對背景膨脹

程式 10-2 侵蝕與膨脹

```
import cv2, numpy as np

binary = cv2.imread("binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

eroded  = cv2.erode(binary, kernel, iterations=1)    # 侵蝕:縮水
dilated = cv2.dilate(binary, kernel, iterations=1)   # 膨脹:長大

# iterations 控制重複次數,等效於放大結構元素
eroded3 = cv2.erode(binary, kernel, iterations=3)

cv2.imwrite("eroded.png", eroded)
cv2.imwrite("dilated.png", dilated)
```

10.2.3 推導：為什麼侵蝕就是取最小值，以及對偶性從哪裡來

10.2.1 節提過一句「對灰階影像而言，侵蝕等同於在鄰域取最小值」，很多人把它當成一個巧合記起來。其實這不是巧合，而是同一個定義在不同值域上的樣子，把它推一遍，型態學的許多性質就會一次說得通。

先把二值的定義寫成集合式。設前景為集合 A，結構元素為 B，侵蝕的定義是「把 B 平移到位置 z，若 B 整個落在 A 之內，就保留 z」:侵蝕(A, B) = 所有「把 B 平

移到該處之後，B 完全落在 A 之內」的位置所成的集合。現在把二值影像看成一個只取 0 或 1 的函數 f，「B 整個落在前景內」等價於「B 覆蓋到的每一個位置 f 都是 1」，也就是「B 覆蓋範圍內 f 的最小值是 1」。把這句話裡的 1 換成任意灰階，就得到灰階版本的侵蝕：輸出等於結構元素覆蓋範圍內輸入的最小值。膨脹的推導完全對稱，只是把「完全包含」換成「有交集」，「最小值」換成「最大值」。圖 10-4 把 cv2.erode 與 scipy 的 minimum_filter 逐像素比對，最大差異是 0——不是相近，是同一個運算。

對偶性則來自一個更簡單的觀察：「B 完全落在 A 內」與「B 完全不碰到 A 的補集」是同一句話。把它寫成式子就是「對 A 侵蝕 = 對 A 的補集膨脹之後再取補集」，其中膨脹所用的結構元素要先翻轉 180 度。若結構元素對原點對稱（方形、十字、橢圓都是），這個翻轉可以省略。圖 10-5 把這條式子以及它的斷開閉合版本都逐像素驗證過，相異的像素數都是 0。這條性質的實用價值是省一半的記憶：你只需要熟悉侵蝕，膨脹就是「對背景做侵蝕」。

順著這條線，還有四條代數律值得記住，因為它們會直接影響你怎麼寫程式。第一，侵蝕與膨脹都是遞增的：A 含於 C 則結果也維持包含關係。第二，膨脹滿足結合律：先用 B1 膨脹再用 B2 膨脹，等於直接用「B1 膨脹 B2」所得到的那個結構元素膨脹一次。這條就是 iterations 參數的理論基礎——用 3×3 的結構元素膨脹兩次，等於用 5×5 膨脹一次。第三，侵蝕與膨脹都可以分解：一個 k×k 的方形結構元素可以拆成「1×k 的水平」接「k×1 的垂直」，運算量由 k^2 降到 2k——這正是第 5.3.3 節可分離濾波的型態學版本。第四，侵蝕對交集、膨脹對聯集具有分配律，但反過來只有包含關係而非等號;這個不對稱是後面許多「順序不能換」的根源，10.3.3 節會接著談。

最後是一個實務推論。既然灰階侵蝕就是取最小值，那麼它對「鹽」（白點）特別有效、膨脹對「胡椒」（黑點）特別有效。但圖 10-4 的實測提醒我們不要只做一半：對含鹽雜訊的影像只做侵蝕，RMSE 反而由 20.9 升到 20.9 以上，因為白點雖然消失

了，整張影像也跟著被壓暗。要做完整的斷開（蝕完再脹回來）才降到 10.1，而這個 10.1 幾乎等於「對一張完全乾淨的影像做同樣斷開」的 10.1——也就是說雜訊已經被清光，剩下的全是運算本身的平滑代價。這也是為什麼下一節要談組合運算。

10.3 兩個組合運算：斷開與閉合

把侵蝕與膨脹依不同順序組合，得到兩個極為實用的運算。它們的巧妙之處在於：能在「去除雜訊/填補缺陷」的同時，大致維持物件的原始大小——因為一縮一脹相互抵消。

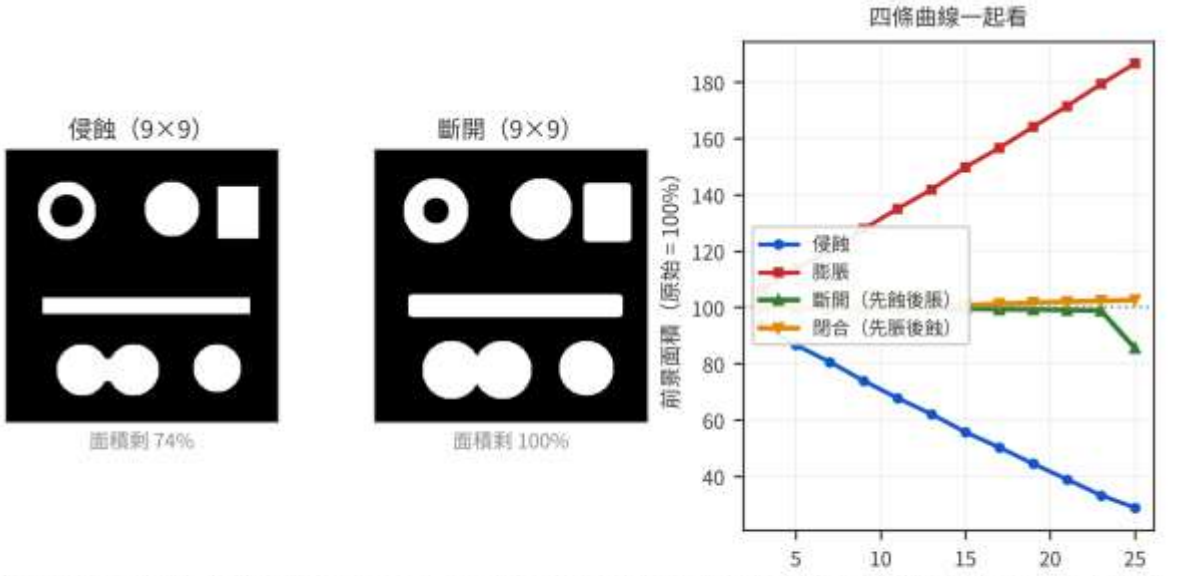


圖 10-6　一縮一脹為什麼能保住物件原本的大小

10.3.1 斷開

斷開(Opening)是「先侵蝕、後膨脹」。先侵蝕消滅了小於結構元素的雜點與細頸，後續的膨脹再把倖存的物件恢復到接近原本的大小。淨效果是：去除背景中的孤立小白點、平滑物件的外部輪廓、斷開細頸相連的物件——而主體幾乎不變。口訣：斷開「開」掉背景的雜訊。

10.3.2 閉合

閉合(Closing)是「先膨脹、後侵蝕」。先膨脹填補了小於結構元素的孔洞與裂縫，後續的侵蝕再把物件收回原本的大小。淨效果是：填補前景內的小孔洞、連接鄰近的斷裂、平滑物件的內部輪廓——而主體幾乎不變。口訣：閉合「閉」合前景的破洞。

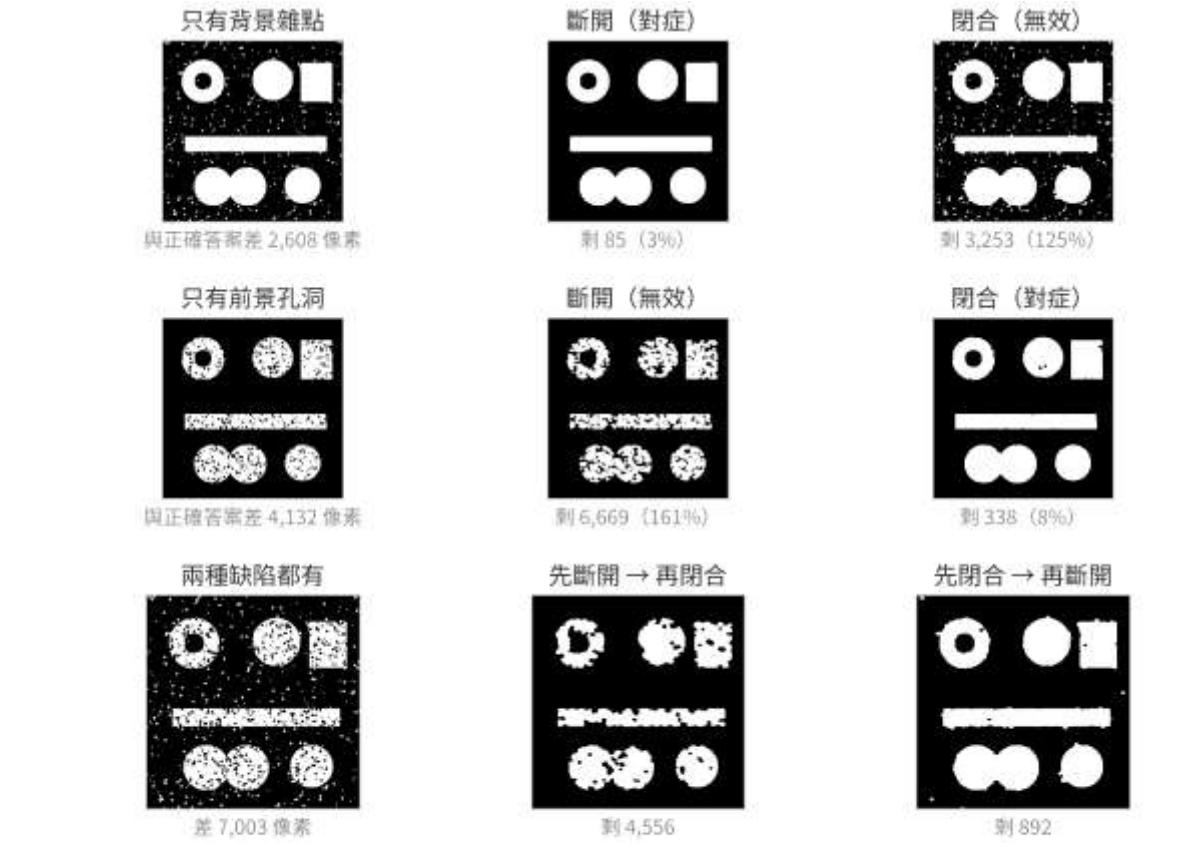
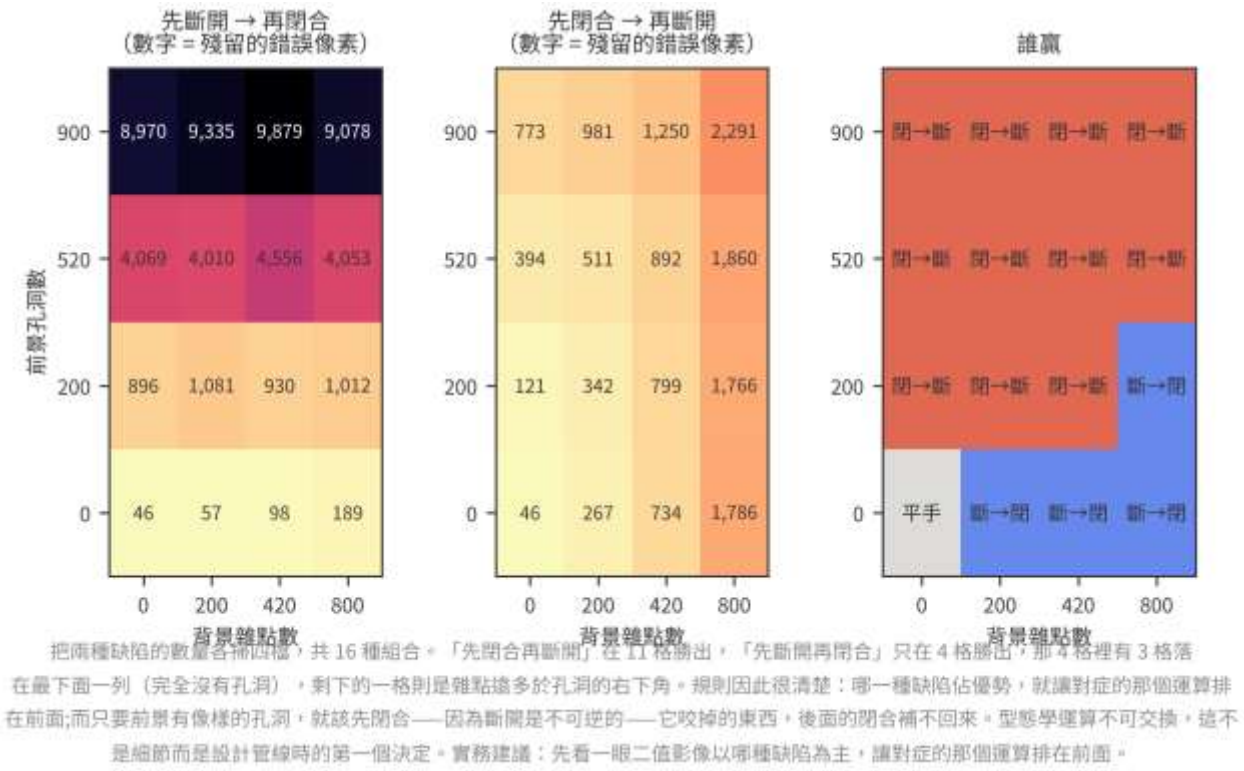


圖 10-7　斷開與閉合各治各的病，而且順序會改變結果

斷開與閉合是實務上最常用的型態學運算，尤其在第 9 章分割之後的清理階段。標準流程往往是：Otsu 二值化 → 斷開去除背景雜點 → 閉合填補前景孔洞 → 連通元件標記計數。第 9 章程式 9-4 的分水嶺前處理，用的正是這套組合。



把兩種缺陷的數量各補四格，共 16 種組合。「先閉合再斷開」在 11 格勝出，「先斷開再閉合」只在 4 格勝出，那 4 格裡有 3 格落在最下面一列（完全沒有孔洞），剩下的一格則是雜點遠多於孔洞的右下角。規則因此很清楚：哪一種缺陷佔優勢，就讓對症的那個運算排在前面;而只要前景有像樣的孔洞，就該先閉合——因為斷開是不可逆的——它咬掉的東西，後面的閉合補不回來。型態學運算不可交換，這不是細節而是設計管線時的第一個決定。實務建議：先看一眼二值影像以哪種缺陷為主，讓對症的那個運算排在前面。

圖 10-8　順序之爭：哪一種缺陷多，就先治哪一種

程式 10-3 斷開與閉合

```
import cv2, numpy as np

noisy = cv2.imread("noisy_binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

opening = cv2.morphologyEx(noisy, cv2.MORPH_OPEN,  kernel)  # 去背景雜點
closing = cv2.morphologyEx(noisy, cv2.MORPH_CLOSE, kernel)  # 填前景孔洞

# 實務常見:先斷開再閉合,一次清乾淨
clean = cv2.morphologyEx(noisy, cv2.MORPH_OPEN, kernel)
clean = cv2.morphologyEx(clean, cv2.MORPH_CLOSE, kernel)

cv2.imwrite("opening.png", opening)
cv2.imwrite("closing.png", closing)
```

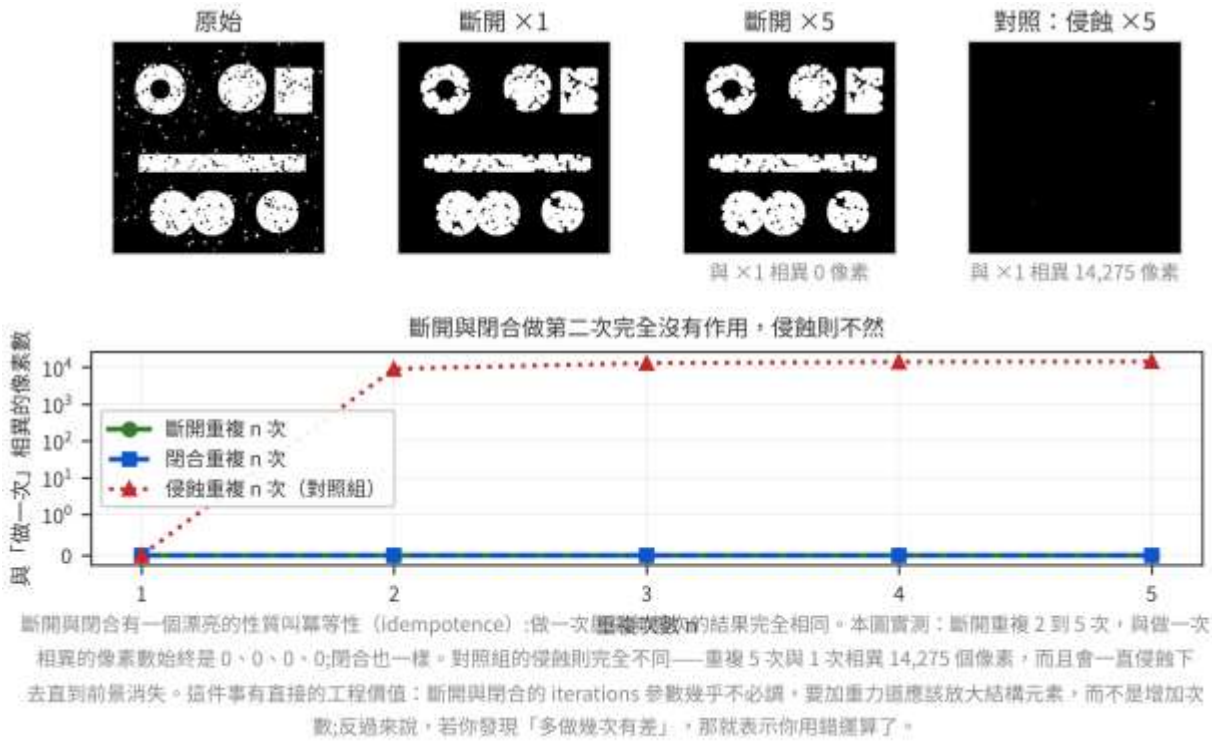


圖 10-9　冪等性：斷開與閉合做第二次完全沒有作用

10.3.3 推導：冪等性，以及為什麼順序不能交換

斷開與閉合不只是「侵蝕加膨脹」的隨意組合，它們滿足兩條很強的性質，而這兩條性質決定了你該怎麼用它們。第一條是冪等性，第二條是不可交換性——前者讓你少調一個參數，後者逼你多想一個決定。

先看冪等性。斷開的定義是：先對 A 用 B 侵蝕，再把結果用 B 膨脹。它有三個基本性質：結果一定含於原影像（遞減性）、原影像變大結果也變大（遞增性）、以及最關鍵的「做兩次等於做一次」——對斷開的結果再斷開一次，結果不變。直觀的理解是：斷開的結果可以被刻畫成「所有能完整塞進 A 裡的 B 的聯集」。既然輸出已經是這些 B 的聯集，再斷開一次時，每一個 B 當然仍然塞得進去，於是什麼都不會改變。閉合的推導對稱，只是把「塞進前景」換成「塞進背景」。

圖 10-9 把這件事實測出來：斷開重複 2、3、4、5 次，與做一次相異的像素數全部都是 0;閉合也一樣。對照組的侵蝕則完全不同——重複五次與一次相差一萬四千多個像素，而且會一直侵蝕到前景消失。工程上的結論很直接：斷開與閉合的

iterations 參數不必調，設 1 就好;要加重力道請放大結構元素，而不是增加次數。反過來說，若你發現「多做幾次結果有差」，那就表示你呼叫的其實不是斷開或閉合（例如用錯成 erode）。

再看不可交換性，這是更容易出事的一條。斷開再閉合與閉合再斷開是兩個不同的運算，而課本常寫的「先斷開再閉合，一次清乾淨」只在某些情況下成立。圖 10-8 把兩種缺陷的數量各掃四檔、共十六種組合實測：「先閉合再斷開」在十一格勝出，「先斷開再閉合」只在四格勝出，而那四格裡有三格是「完全沒有孔洞」的情形。原因不難理解：斷開是遞減運算，它咬掉的東西後面補不回來。若前景布滿孔洞，先斷開會把物件咬成碎片，之後的閉合只能把碎片連成更奇怪的形狀;反過來先閉合把物件補成實心，再斷開就能乾淨地掃掉背景雜點。

把這兩條性質合起來，得到一條可以直接寫進規格書的原則：先看一眼你的二值影像以哪一種缺陷為主，讓對症的那個運算排在前面;只要前景有像樣的孔洞，就先閉合。順序是設計管線時的第一個決定，不是最後一個微調。順帶一提，把斷開與閉合交替使用、並逐次放大結構元素的作法有個名字叫交替序列濾波（Alternating Sequential Filter），在雜訊尺度分布很廣的場合比單次運算穩健得多，代價是運算時間隨級數線性增加。

10.4 進階型態學運算

10.4.1 型態學梯度、頂帽與黑帽

1. 型態學梯度(Gradient)：膨脹減去侵蝕，得到物件的輪廓。它是另一種邊緣偵測方式，與第 9 章的 Sobel、Canny 殊途同歸，但基於形狀運算而非微分。

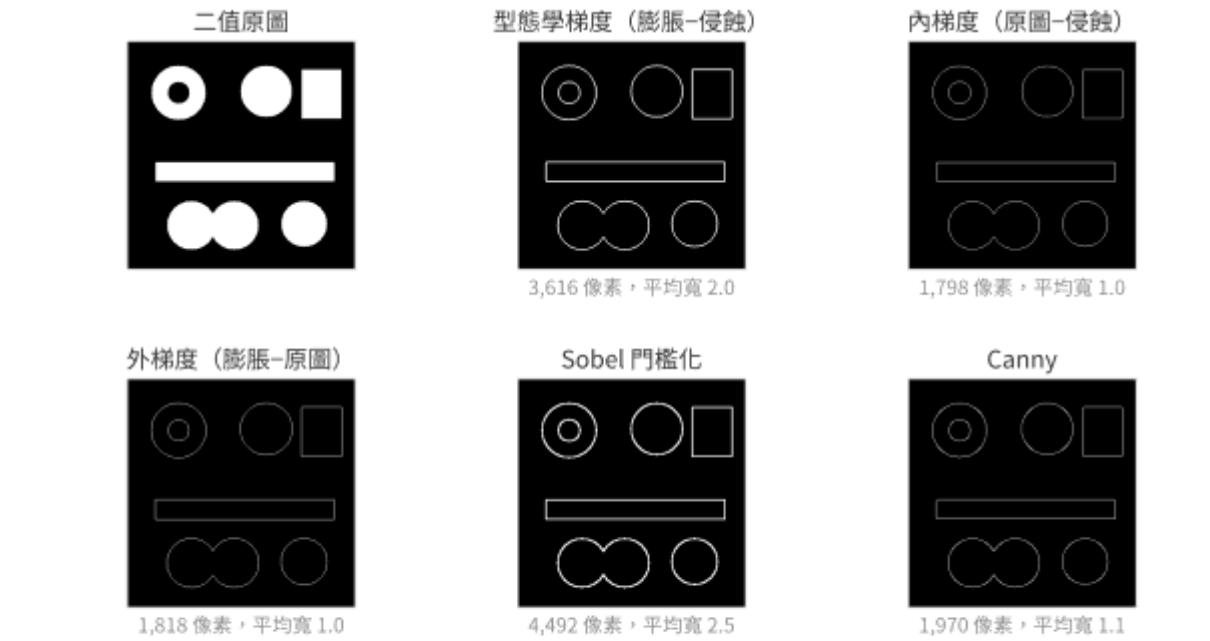


圖 10-10　型態學梯度：內梯度、外梯度與微分法的對照

2. 頂帽轉換(Top Hat)：原圖減去斷開的結果，萃取出「比周圍背景亮、且小於結構元素」的細節。它的殺手級應用是校正不均勻的背景照明——這正是第 8 章「洋裝效應」與第 9 章打光不均問題的另一種解法：用一個大的結構元素做斷開，得到的是緩慢變化的背景照明，原圖減去它，就抽出了均勻照明下的前景細節。

3. 黑帽轉換(Black Hat)：閉合的結果減去原圖，萃取出「比周圍背景暗、且小於結構元素」的細節，例如亮背景上的暗刮痕、暗字。

頂帽與黑帽在 AOI 檢測中極為實用：當工件表面因曲面或打光而有漸變的亮度背景，直接門檻分割會失敗，先做頂帽或黑帽拉平背景，瑕疵便無所遁形。這與第 6.3.3 節的同態濾波是解決同一問題(不均勻照明)的不同途徑——一個在空間域用形狀運算，一個在頻率域用對數濾波。

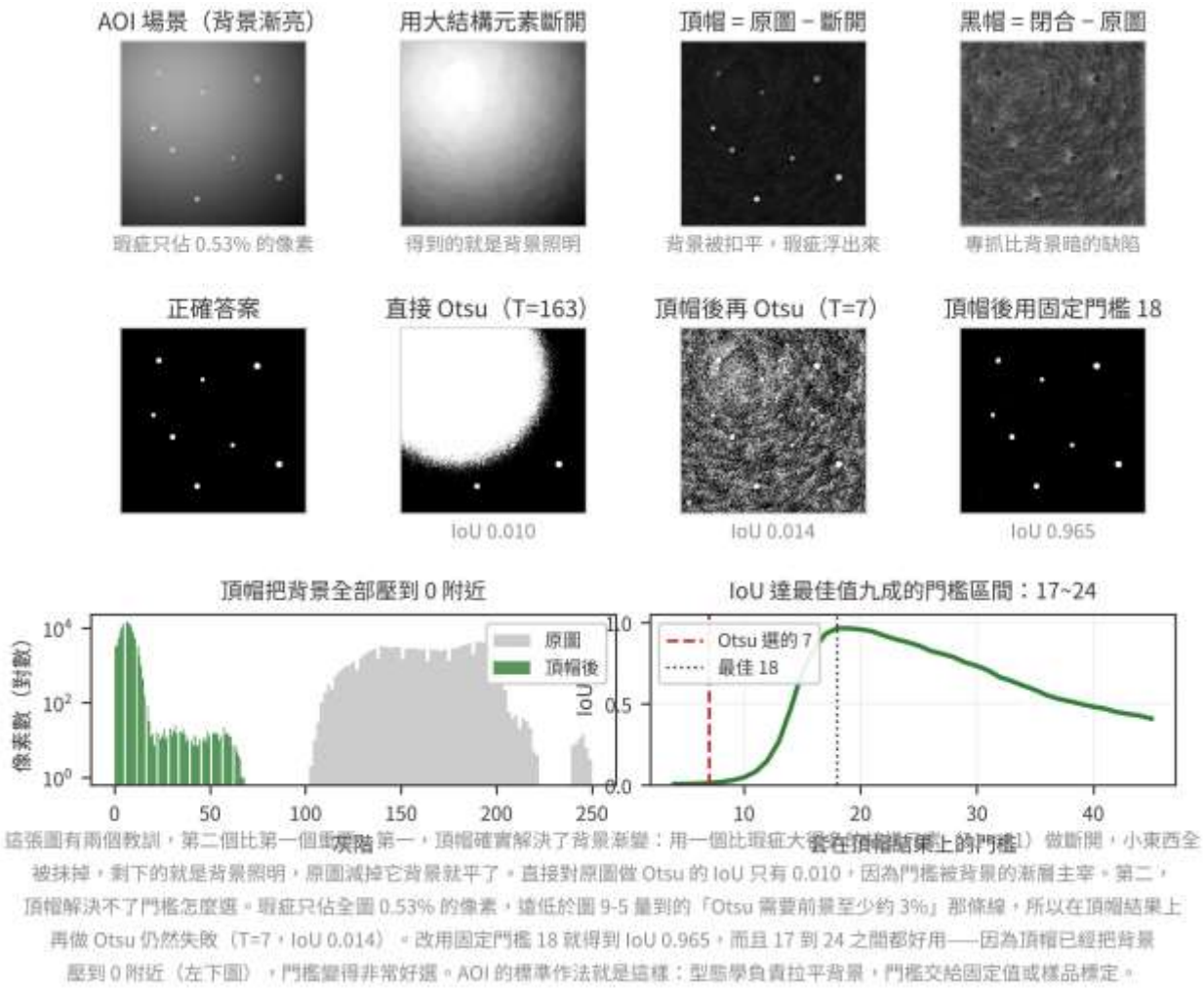


圖 10-11　頂帽拉平了背景，但門檻還是得自己選

10.4.2 Hit-or-Miss、細線化與骨架化

Hit-or-Miss 轉換是型態學的形狀偵測工具：它同時檢查「前景該在的地方是否是前景」與「背景該在的地方是否是背景」，只有兩者都吻合才命中。它能偵測特定的局部形狀，例如角點、線的端點、孤立點，是細線化與骨架化的理論基礎。

細線化(Thinning)反覆剝除物件的邊界像素，但保留其連通性與端點，最終把物件細化成單像素寬的線。骨架化(Skeletonization)則把物件化約成其「中軸」——想像從物件的每個邊界點同時向內燃燒，火線相遇熄滅之處的軌跡就是骨架。這兩者在指紋辨識(把指紋脊線細化)、文字辨識(把筆畫細化以分析結構)、電路板走線分析、道路網路萃取中不可或缺。

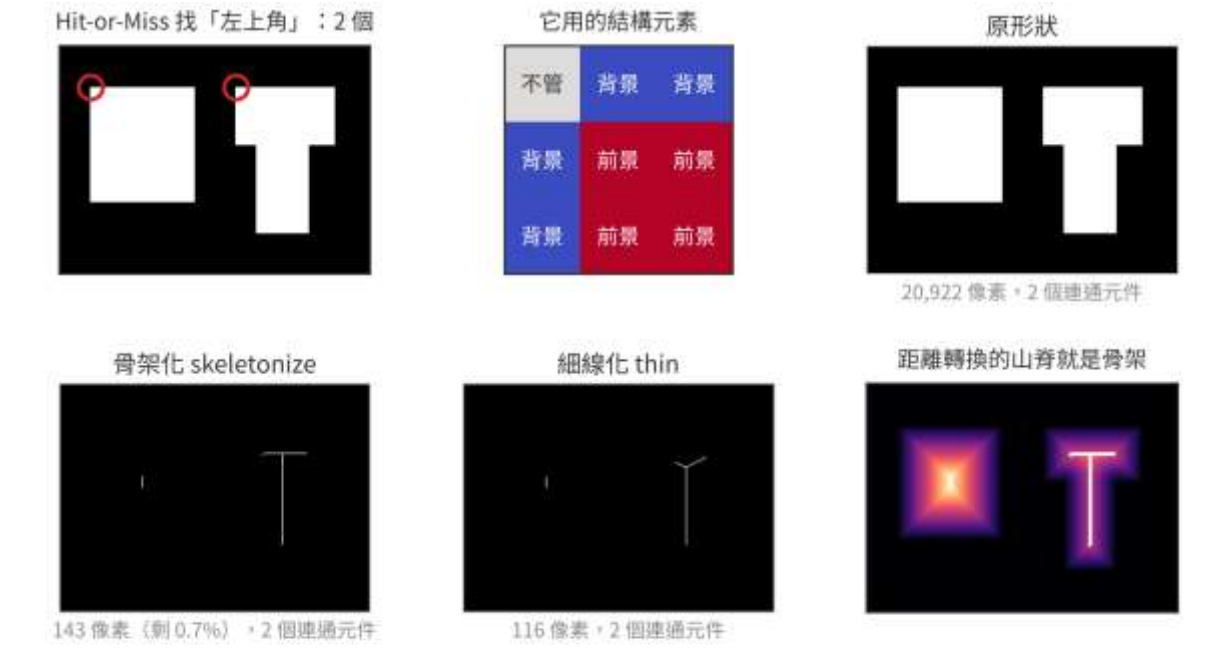
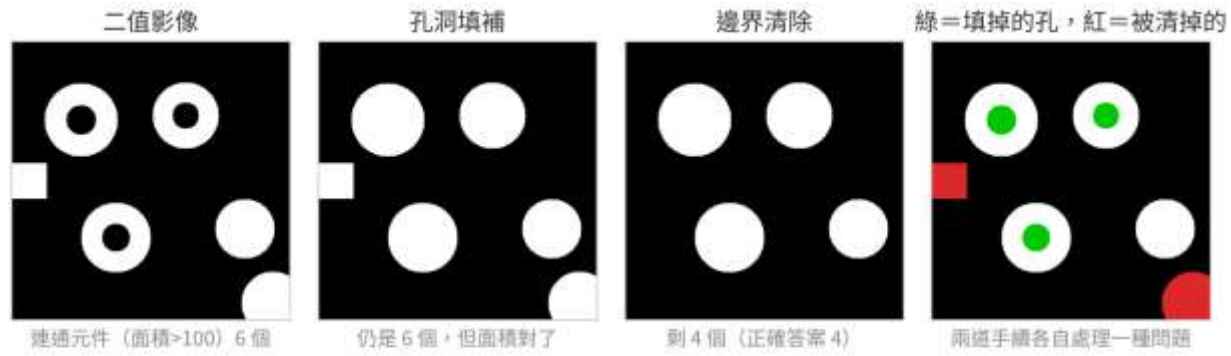


圖 10-12　Hit-or-Miss 找形狀，骨架化保住拓撲

10.4.3 孔洞填補與邊界清除

孔洞填補(Hole Filling)以型態學重建的方式，把前景物件內部完全封閉的背景區域(孔洞)填成前景。它在「數清楚有幾個實心物件」時很重要——一個有孔洞的墊圈，若不填補孔洞，連通元件標記可能會被孔洞干擾。邊界清除(Border Clearing)則移除接觸到影像邊界的物件，因為這些物件通常是「被畫面切到、資訊不完整」的，在量測與計數時應予排除——例如量測顯微鏡下的細胞尺寸時，只計算完整落在視野內的細胞。



計數之前通常要先做兩件事。孔洞填補把物件內部封閉的背景補成前景:本圖的墊圈若不填補，量到的面積是 4,624 像素，填補後是 5,525，差了 16%——若你的驗收標準是面積，這個誤差足以讓良品被判成不良。邊界清除則丟掉所有碰到影像邊緣的物件，因為它們被畫面切掉了一部分，面積與形狀都不可信;做完之後連通元件由 6 個降到 4 個，與正確答案 4 一致。順序是固定的：先填補再清除。反過來做，碰邊的物件會先被清掉，它內部的孔洞反而留下來變成假物件。

圖 10-13　孔洞填補與邊界清除：計數之前的兩道手續

10.5 連通元件標記與距離轉換

10.5.1 連通元件標記

連通元件標記(Connected Component Labeling)是把二值影像中「每一個相連的前景區域」各自賦予一個獨立編號的過程——這正式回答了「畫面中有幾個物件」這個問題。它建立在第 3 章介紹的連通性概念上：採用 4-連通或 8-連通，計數結果可能不同(第 3 章已強調過此點)。

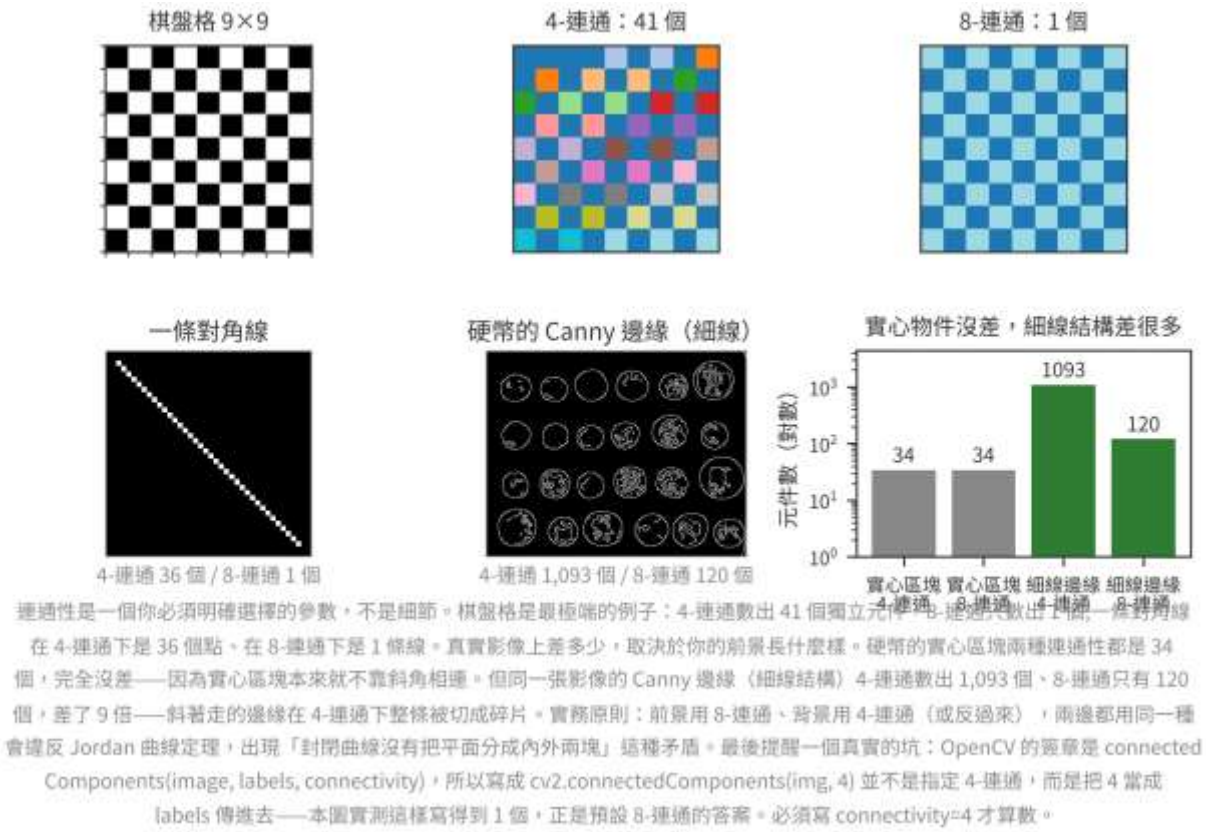


圖 10-14　4-連通還是 8-連通，細線結構上差了九倍

OpenCV 的 connectedComponentsWithStats 不僅標記，還一併回傳每個元件的統計資訊：面積(像素數)、外接矩形(位置與大小)、質心座標。這一步直接把「一堆白色像素」轉化為「一份物件清單」，是所有計數與量測應用的核心——第 9 章的零件計數、細胞計數，骨幹都在這裡。搭配面積門檻，還能輕鬆濾除殘餘的小雜點(面積過小者不計)。

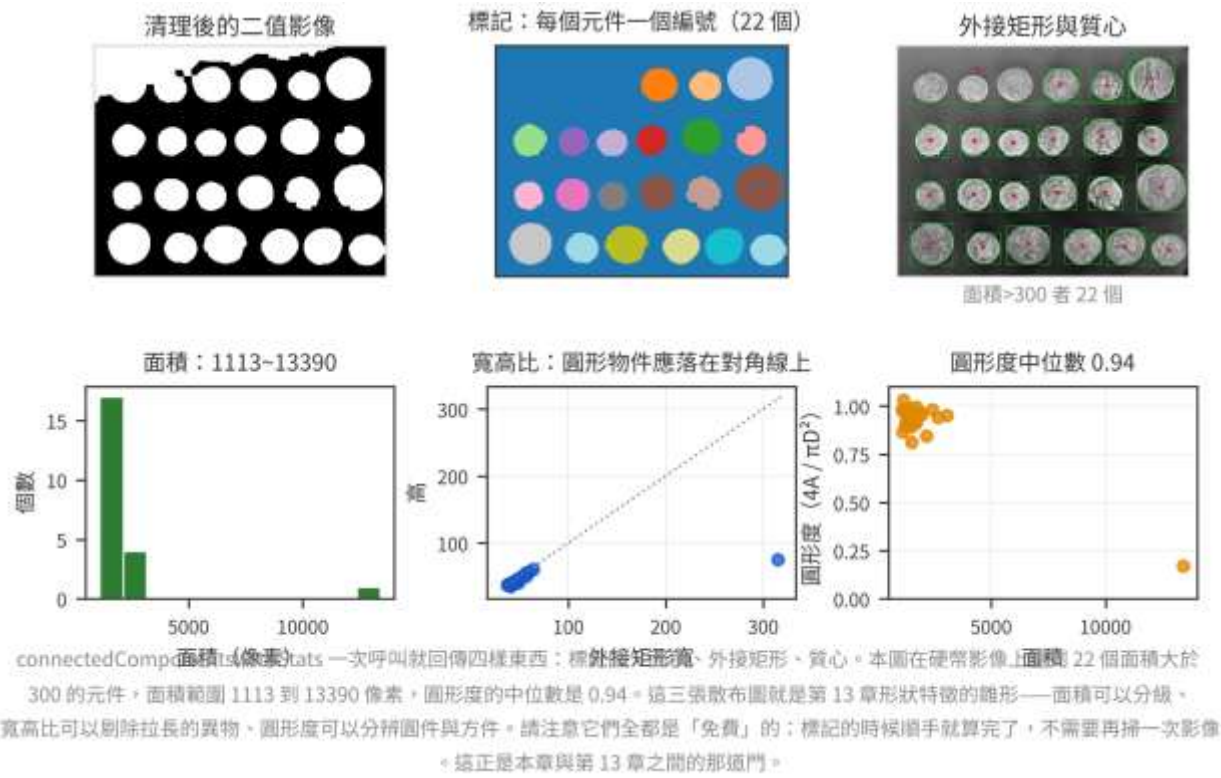


圖 10-15　連通元件標記一次給你的四樣東西

10.5.2 距離轉換

距離轉換(Distance Transform)為二值影像中的每一個前景像素，計算它到最近的背景像素的距離。結果是一張灰階圖：物件中心離背景最遠(值最大、最亮)，邊界處值最小(接近零)。它用到的正是第 3 章介紹的距離度量(歐氏、D4、D8)。

距離轉換有兩個關鍵應用。其一是求骨架：距離轉換的「山脊」(局部最大值的連線)正是物件的中軸，提供了骨架化的另一條途徑。其二是分離相黏物件：這正是第 9 章程式 9-4 分水嶺分割的核心步驟——對兩個相黏的圓形物件做距離轉換，每個圓的中心會各自形成一個亮點(局部最大)，以此作為分水嶺的「確定前景」標記，就能把黏在一起的物件正確分開。至此，第 9 章分水嶺流程中那段「距離轉換」的程式碼，其原理便完全清晰了。

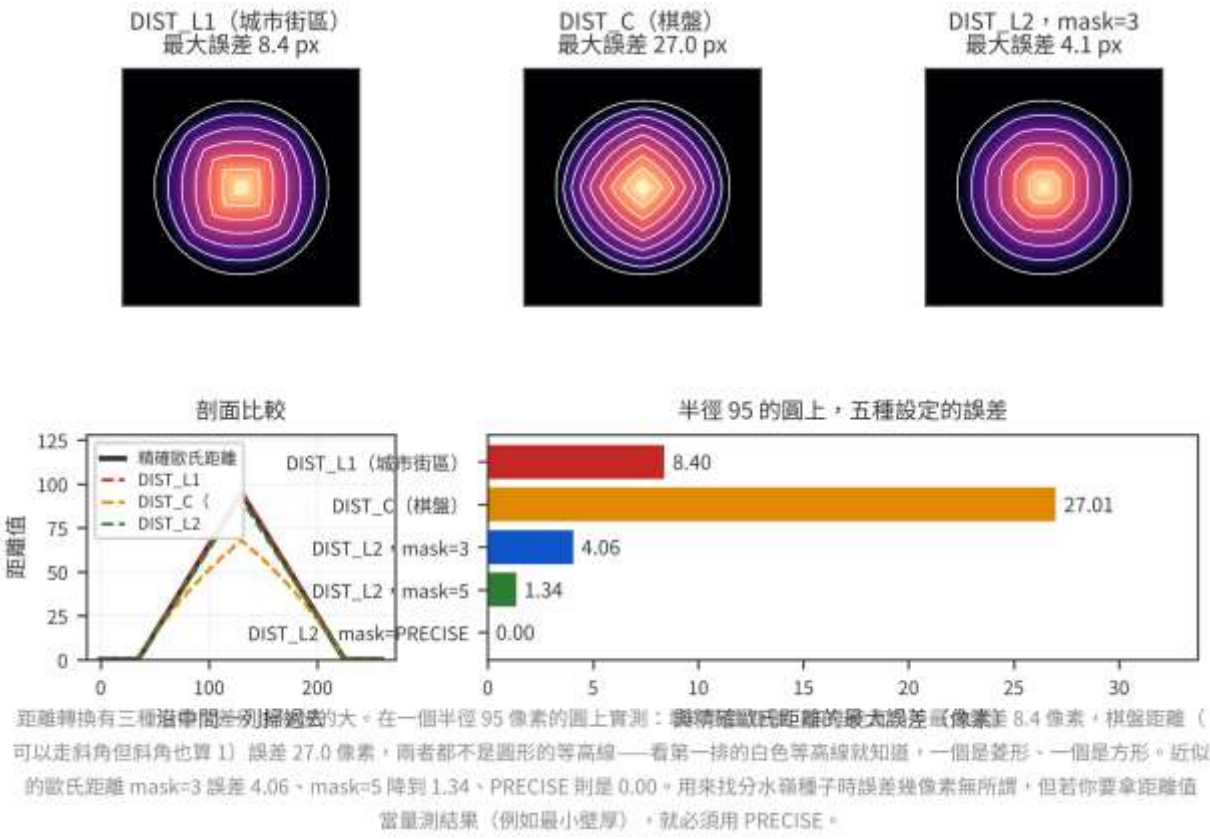


圖 10-16　距離轉換的三種度量：等高線是菱形、方形還是圓形

程式 10-4 連通元件標記與距離轉換

```
import cv2, numpy as np

binary = cv2.imread("parts.png", cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(binary, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 連通元件標記 + 統計 ---
n, labels, stats, centroids = cv2.connectedComponentsWithStats(binary)
print("物件數(扣除背景):", n - 1)

output = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
for i in range(1, n):                          # 0 是背景,跳過
    area = stats[i, cv2.CC_STAT_AREA]
    if area < 50:                              # 面積門檻:濾除雜點
        continue
    x = stats[i, cv2.CC_STAT_LEFT]
    y = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]
    cx, cy = centroids[i]
    cv2.rectangle(output, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.circle(output, (int(cx), int(cy)), 3, (0, 0, 255), -1)
    print(f"物件 {i}: 面積={area}, 質心=({cx:.1f}, {cy:.1f})")

# --- 距離轉換(分離相黏物件的關鍵) ---
```

```
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
dist_vis = cv2.normalize(dist, None, 0, 255,
                         cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("distance.png", dist_vis)
cv2.imwrite("labeled.png", output)
```

10.5.3 推導：連通元件標記怎麼只掃兩遍就完成

「把每一個相連的前景區域各給一個編號」聽起來像是要反覆搜尋，但標準演算法只掃描影像兩遍，而且與物件的數量、形狀都無關。它用到的資料結構叫聯集-尋找（Union-Find），是演算法課的經典題目，也是理解「為什麼計數這麼快」的關鍵。

第一遍由上到下、由左到右掃過每個前景像素，只看它「已經掃過的鄰居」——以 8-連通為例，就是左、左上、上、右上四個。分三種情況處理：若四個鄰居都是背景，給這個像素一個全新的標籤;若鄰居只有一種標籤，就沿用它;若鄰居出現兩種以上不同的標籤，表示這幾個原本以為不同的區域其實是同一個，於是沿用其中最小的，並把「這些標籤是等價的」記錄下來。第二遍再掃一次，把每個像素的暫時標籤換成它所屬等價類的代表。

等價關係的記錄就是 Union-Find 做的事。它用一個陣列存每個標籤的「父節點」，find 沿著父節點往上找到根，union 把兩棵樹接起來。若只做這兩件事，最壞情況會退化成一條長鏈;但加上兩個標準優化——路徑壓縮（find 的時候把沿途節點直接接到根上）與按秩合併（永遠把矮的樹接到高的樹下）——之後，m 次操作的總成本是 $O(m\,\alpha(n))$，其中 α 是反阿克曼函數，在任何實際會遇到的輸入規模下都小於 5，可以當成常數。於是整個演算法的複雜度就是 $O(N)$，N 是像素數。

這個「單次掃描順手把統計算完」的特性，正是 connectedComponentsWithStats 能夠免費回傳面積、外接矩形與質心的原因。面積是計數、外接矩形是四個極值、質心是座標的累加和除以面積——三者都可以在第一遍掃描時邊走邊累加，合併等價類時再把兩份統計加起來即可，完全不需要再掃一次影像。圖 10-15 用這些「免費」的統計量畫出三張散布圖，那其實已經是第 13 章

形狀特徵的雛形了。

最後是兩個必須提醒的實務細節。第一，連通性一定要明寫。OpenCV 的簽章是 connectedComponents(image, labels, connectivity)，所以寫成 cv2.connectedComponents(img, 4) 並不是指定 4-連通，而是把 4 當成 labels 參數傳進去，連通性仍然是預設的 8——圖 10-14 實測這個寫法得到的正是 8-連通的答案。必須寫成 connectivity=4 才算數，而且這個錯誤不會報錯。第二，前景與背景應該用不同的連通性（前景 8、背景 4，或反過來）。兩邊都用同一種會違反 Jordan 曲線定理，出現「一條封閉曲線沒有把平面分成內外兩塊」這種矛盾。圖 10-14 也量到，同一張硬幣影像的實心區塊兩種連通性都是 34 個，但它的 Canny 邊緣圖在 4-連通下是 1,093 個、8-連通下只有 120 個——差了九倍。前景愈細，這個選擇愈要命。

10.6 Haar 級聯分類器與人臉偵測

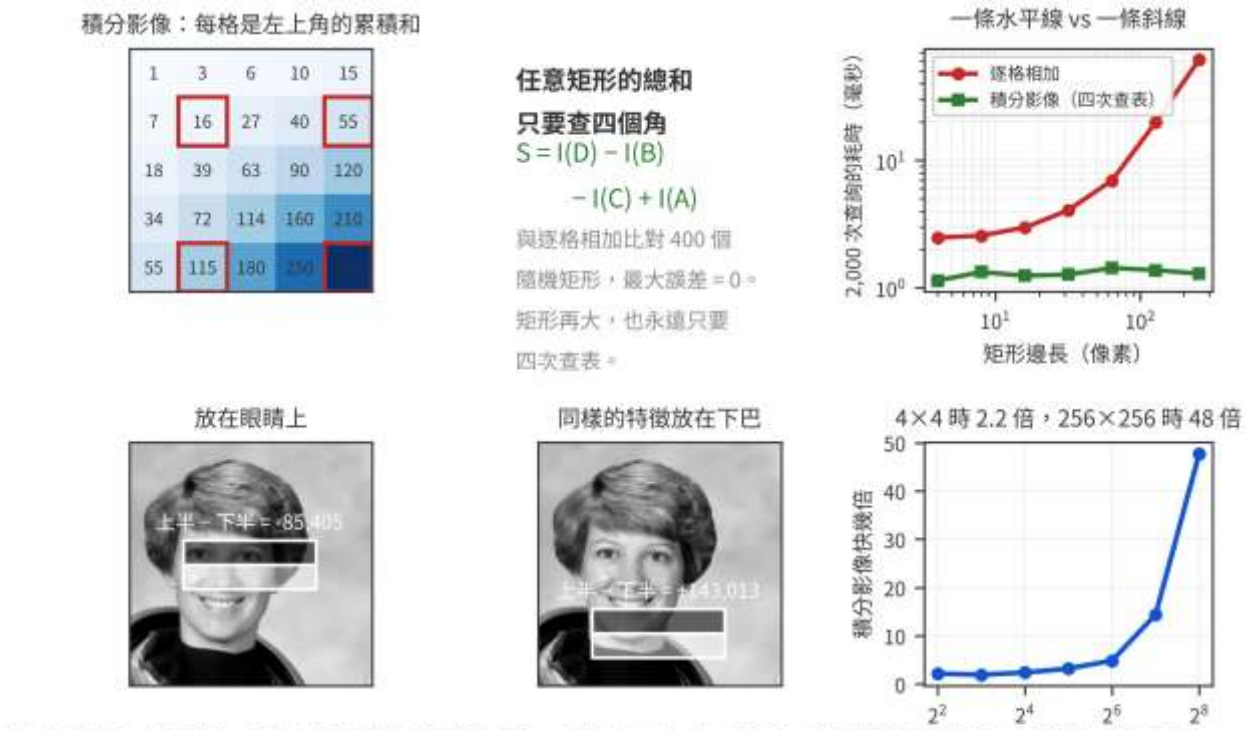
本節把叫獸開講的 Viola-Jones 演算法落實成可執行的技術，它也是本章「二值/區域思維」通往第 15、16 章「機器學習物件偵測」的橋梁。

10.6.1 三大支柱回顧

承叫獸開講，Viola-Jones 由三個技術支柱撐起。第一，Haar-like 特徵：計算相鄰矩形區域的亮度差，捕捉人臉的明暗規律(如眼睛比臉頰暗、鼻樑比兩側亮)。第二，積分影像：預先計算每點左上方所有像素的累加和，使任意矩形區域的總和只需查表三次加減即可求得，與區域大小無關——這是它能高速計算海量特徵的關鍵。第三，AdaBoost 與級聯：一個偵測視窗中可能的 Haar 特徵多達十八萬個，AdaBoost 從中篩選出最有鑑別力的少數;再把分類器串成級聯，前段快速淘汰非人臉區域，後段精細確認，達成即時偵測。

值得一提的是，積分影像的技巧與本章 10.5 節連通元件標記「單次掃描完成統計」的精神相通，都是「用預先計算換取查詢時的常數時間」——這是演算法設計中極重

要的空間換時間思維，學生應細細體會。



積分影像的每一格存的是「從左上角到這裡的所有像素總和」，建表只要掃一遍。有了它，任意矩形的總和都只要查四個角做三次加減——本圖以 400 個隨機矩形驗證，與逐格相加的最大誤差是 0。代價是固定的，所以矩形愈大賺愈多：邊長 4 時快 2.2 倍，邊長 256 時快 48 倍。Haar-like 特徵就是幾個矩形總和的加減。本圖的雙矩形特徵問的是「上半比下半暗多少」：放在眼睛帶上得到 -85,405（眼窩比額骨暗，所以是負的），同一個特徵原封不動移到下巴卻是 +143,013（正的）。一正一負，光靠這一個特徵就能排除掉不少位置——這就是級聯第一關在做的事。這個「預先累加換取查詢時的常數時間」的手法，與 10.5 節連通元件「單次掃描完成統計」是同一種思維。

圖 10-17　Haar-like 特徵與積分影像：矩形再大也只要四次查表

10.6.2 OpenCV 實作

OpenCV 內建訓練好的 Haar 級聯模型(XML 檔)，涵蓋正面人臉、眼睛、微笑、全身等，可直接載入使用。

程式 10-5 Haar 級聯人臉與眼睛偵測

```
import cv2

# 載入 OpenCV 內建的正面人臉 Haar 級聯模型
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_eye.xml")

img = cv2.imread("group_photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)   # 只需灰階

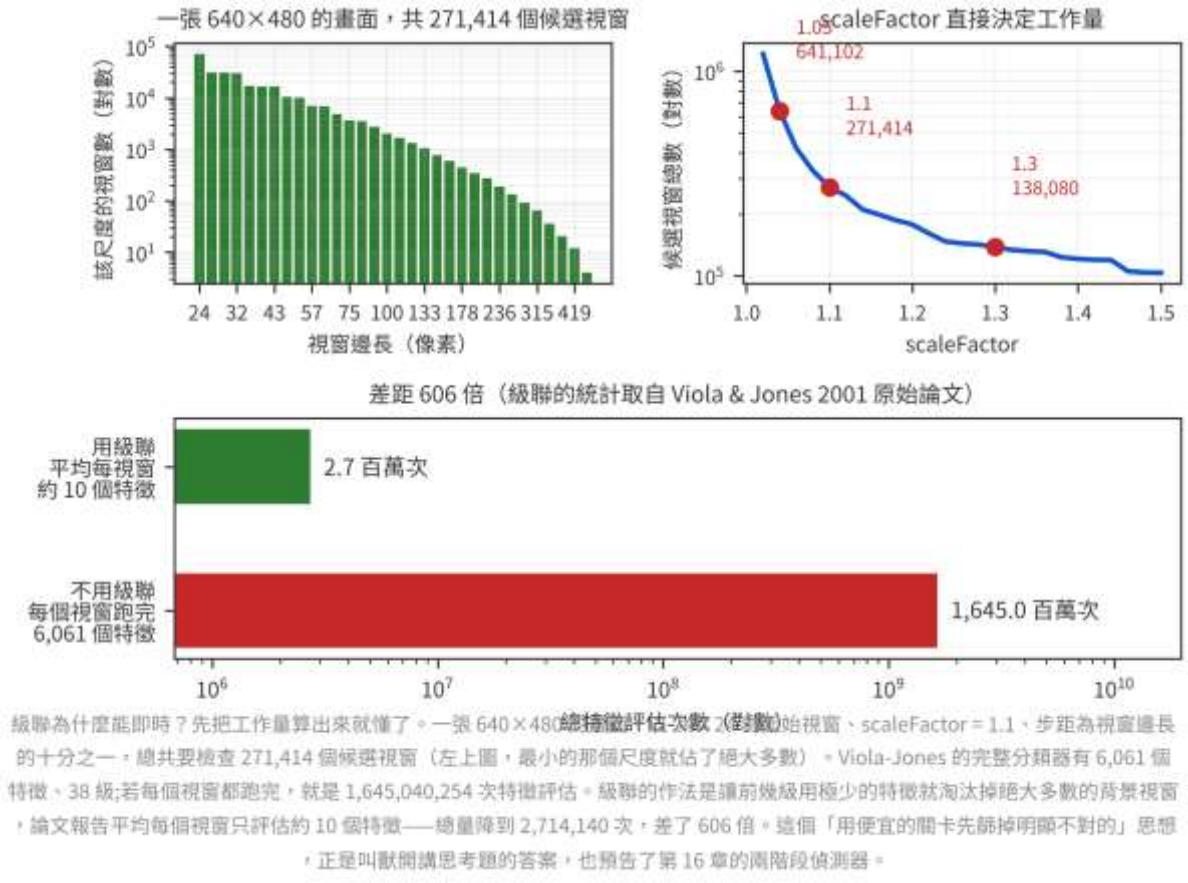
# detectMultiScale:在多種尺度下滑動視窗偵測
faces = face_cascade.detectMultiScale(gray,
```

```
            scaleFactor=1.1,      # 每次縮放比例(影響速度與漏檢)
            minNeighbors=5,       # 需多少鄰近偵測才算數(抑制誤報)
            minSize=(30, 30))     # 最小人臉尺寸

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    roi_gray = gray[y:y+h, x:x+w]              # 第 2 章的 ROI
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(img[y:y+h, x:x+w], (ex, ey),
                      (ex+ew, ey+eh), (255, 0, 0), 2)

print("偵測到人臉數:", len(faces))
cv2.imwrite("faces_detected.jpg", img)
```

兩個參數的調校心法：scaleFactor 愈接近 1，搜尋的尺度愈細密、愈不易漏檢，但速度愈慢;minNeighbors 愈大，愈能抑制誤報(要求同一位置有多個尺度都偵測到才確認)，但也可能漏掉真實人臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 的雙門檻、第 16 章物件偵測的信心門檻，是同一類工程取捨。



級聯為什麼能即時？先把工作量算出來就懂了。一張 640×480 的畫面，scaleFactor = 1.1、步距為視窗邊長的十分之一，總共要檢查 271,414 個候選視窗（左上圖，最小的那個尺度就佔了絕大多數）。Viola-Jones 的完整分類器有 6,061 個特徵、38 級;若每個視窗都跑完，就是 1,645,040,254 次特徵評估。級聯的作法是讓前幾級用極少的特徵就淘汰掉絕大多數的背景視窗，論文報告平均每個視窗只評估約 10 個特徵——總量降到 2,714,140 次，差了 606 倍。這個「用便宜的關卡先篩掉明顯不對的」思想，正是叫獸開講思考題的答案，也預告了第 16 章的兩階段偵測器。

圖 10-18　級聯為什麼能即時：先把候選視窗數算出來

10.6.3 三種辨識架構與 ESP32-CAM 部署

回顧第 2 章介紹的三種 AI 辨識架構——端上運算、雲端運算、近端邊緣運算——Viola-Jones 的輕量特性(不需 GPU、記憶體需求小)使它成為端上運算的理想選擇。事實上，ESP32-CAM 的官方韌體就內建了人臉偵測與辨識功能，正是基於這類輕量演算法。

這帶出一個重要的部署思考：在 ESP32-CAM 這樣算力極為有限的裝置上，深度學習的 YOLO(第 16 章)難以即時執行，而 Viola-Jones 這類傳統方法反而遊刃有餘。這再次印證第 9 章的判斷原則：方法沒有絕對優劣，只有適不適合。當任務是「在低功耗感測節點上偵測正面人臉」時，一個 2001 年的演算法可能比 2020 年代的深度模型更務實。第 17 章會深入這個部署決策。



圖 10-19　OpenCV 5 的現實：Haar 級聯的 API 已經不在了

10.6.4 現況提醒：程式 10-5 需要 OpenCV 4.x

這一節有一個必須講清楚的版本問題。本書插圖的產生環境是 OpenCV 5.0.0（opencv-python 5.0.0.93），在這個版本上實測：cv2.CascadeClassifier 這個類別已經不存在，cv2.data.haarcascades 指向的目錄裡也不再附帶任何 XML 模型檔，連

cv2.HOGDescriptor（行人偵測的傳統作法）也一併消失了。也就是說，程式 10-5 照著打會直接拋出 AttributeError，而不是跑出一個空的偵測結果。圖 10-19 把實測的結果整理成一張表。

要怎麼辦，看你的目的。若你要照著程式 10-5 把 Haar 級聯跑一遍（建議初學者這樣做，因為它最能體會 detectMultiScale 兩個參數的影響），請安裝 OpenCV 4.x:pip install "opencv-python<5"。Haar 級聯與內建的 XML 模型在 4.x 中都還在，ESP32-CAM 端上的函式庫也多半是以 4.x 為基礎移植的，所以 Lab 10-3 完全不受影響。

若你用的是現行版本而目的只是「把人臉框出來」，可以改用 cv2.FaceDetectorYN（YuNet）。它是一個很小的卷積網路，模型檔只有數百 KB，需要另外下載;在精度與對側臉、遮擋的容忍度上都明顯勝過 Haar，速度在一般 CPU 上也足以即時。搭配 cv2.FaceRecognizerSF（SFace）還能做人臉特徵比對。這兩個類別在 OpenCV 5 中都還在，本書實測可用。

最後要強調的是：本節真正要教的東西不受這個變動影響。積分影像讓任意矩形的總和降為四次查表（圖 10-17）、級聯用便宜的關卡先淘汰絕大多數候選視窗（圖 10-18），這兩個思想不但沒有過時，還直接出現在第 16 章的兩階段偵測器與第 17 章的邊緣部署策略裡。API 會變，演算法的想法不會。這也提醒你一件事：教科書上的程式碼有保存期限，讀程式庫的版本變更紀錄（CHANGELOG）是工程師的基本功，而不是等到跑出錯誤才回頭查。

10.7 應用案例

10.7.1 案例一：PCB 與工件的瑕疵前處理

情境：AOI 系統要檢測印刷電路板上的斷路、短路、缺件。分割(第 9 章)後得到的二值影像，線路邊緣常有毛刺、線路中有雜點造成的假斷點。處理流程：先用斷開去

除毛刺與孤立雜點 → 用閉合填補線路上因反光造成的假斷點 → 用型態學梯度萃取乾淨的線路輪廓 → 用細線化把線路化約成單像素骨架，便於分析走線的連通性與交叉點。若背景照明不均，則在分割前先用頂帽轉換拉平背景。這整套流程展示了型態學作為「分割與量測之間的清理工序」的核心價值。

10.7.2 案例二：細胞與顆粒計數

情境：顯微鏡影像中要自動計數細胞或顆粒，且它們常相互接觸。這是本章與第 9 章的完整綜合：Otsu 二值化 → 斷開去雜點 → 孔洞填補(把細胞內的亮點填實)→ 邊界清除(排除被視野邊緣切到的不完整細胞)→ 對相黏的細胞做距離轉換，取局部極大作為標記 → 分水嶺分割分開它們 → 連通元件標記計數並量測每個細胞的面積。生物醫學、水質檢測、材料科學的顆粒分析，骨幹都是這條管線。

10.7.3 案例三：即時人臉偵測門禁

情境：用 ESP32-CAM 打造一個低成本的門禁裝置，偵測到人臉時觸發後續動作(拍照上傳、開門、記錄)。這是叫獸開講與第 2 章三架構的實戰整合：ESP32-CAM 在端上以 Haar 級聯即時偵測人臉(端上運算，低延遲、保護隱私)，偵測到後再把影像上傳雲端做進一步的身分辨識(雲端運算，借助強大算力)——這正是第 2 章「近端/雲端混合架構」的體現：輕量的偵測放在端上、耗算力的辨識放在雲端，兼顧即時性與準確度。Lab 10-3 會實作這個系統的偵測端。

10.7.4 案例四：車牌字元分割

情境：車牌辨識(第 13 章會完整處理)的前段，需要把車牌上的每個字元切開。流程：定位並裁切出車牌區域 → 轉灰階、Otsu 二值化 → 用閉合連接字元中斷裂的筆畫(如數字 8 因印刷不良而斷開)→ 用斷開去除邊框雜點 → 連通元件標記，每個連通區域就是一個字元 → 依面積、長寬比濾除非字元的雜訊區域 → 依 x 座標排序，得到字元序列。這條「二值化 → 型態學清理 → 連通元件分割字元」的流程，是所有文字/字元辨識(OCR)的共通前處理，也直接銜接第 13 章的特徵擷取與第 15 章的字元分類。



圖 10-20　車牌字元分割：本章工具的綜合演練

10.8 本章與全書的觀念連結總表

相關章節	共用的觀念	二值處理中的角色
第 3 章 數位影像基礎	連通性、距離度量	連通元件標記與距離轉換的理論基礎
第 6 章 頻率域	不均勻照明校正	頂帽轉換與同態濾波是同一問題的兩種解法
第 7 章 影像還原	最大值/最小值濾波	灰階膨脹即最大值濾波、侵蝕即最小值濾波
第 9 章 影像分割	二值化、分水嶺	型態學是分割輸出的直接清理工序;距離轉換供分水嶺標記
第 13 章 特徵擷取	輪廓、形狀特徵、車牌	連通元件提供物件清單;字元分割接特徵擷取

第 15、16 章 深度學習	Haar 級聯、物件偵測	Viola-Jones 是深度學習偵測器的前輩
第 17 章 邊緣部署	三種辨識架構、ESP32-CAM	輕量的 Haar 級聯適合端上運算

表 10-1 第 10 章與全書其他章節的觀念連結

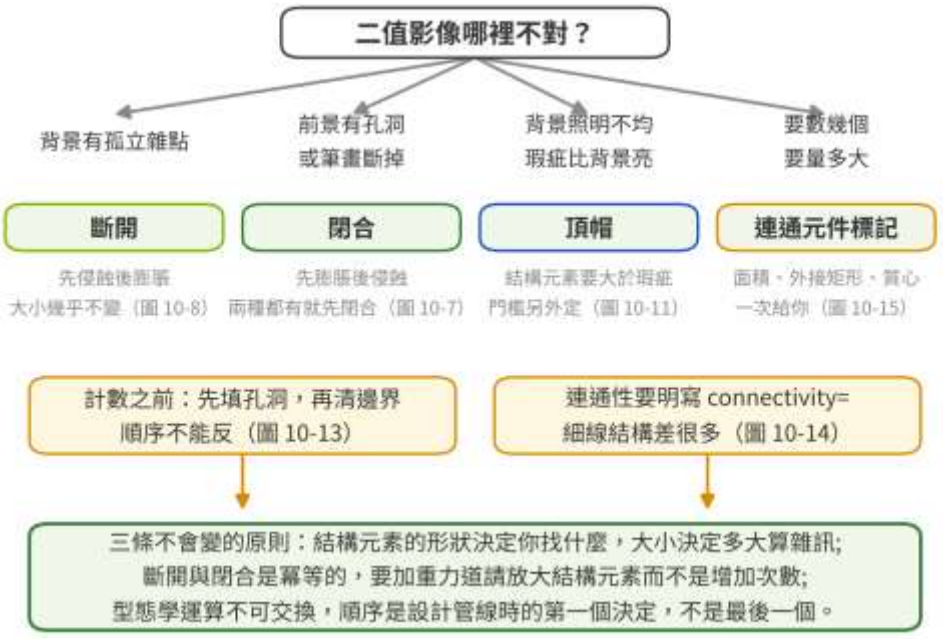


圖 10-21　第 10 章的決策地圖

程式實作 Lab 10

Lab 10-1 型態學運算探索器

目標：建立對型態學運算的直覺。(a)取一張含雜訊的二值影像(可對乾淨二值圖加椒鹽雜訊);(b)以 cv2.createTrackbar 建立可調整結構元素形狀(方/十字/橢圓)、大小、運算類型(侵蝕/膨脹/斷開/閉合)與 iterations 的即時介面;(c)實驗並記錄：去除背景孤立雜點該用哪個運算?填補前景小孔該用哪個?斷開細頸相連的兩物件該用哪個?(d)驗證對偶性：對前景侵蝕，與對背景膨脹(先反轉、膨脹、再反轉)，結果是否相同?(e)驗證與第 7 章的關聯：對灰階影像做膨脹，結果是否等同於最大值濾波?

Lab 10-2 完整計數量測管線

目標：整合第 9、10 章完成一條可用的計數量測系統。(a)拍一張散落的螺絲、鈕扣或豆子照片;(b)依序完成：CLAHE 前處理(第 5 章)→ Otsu 二值化(第 9 章)→ 斷開去雜點 → 閉合填孔 → 孔洞填補 → 邊界清除 → 連通元件標記;(c)以面積門檻濾除雜點，框出每個物件、標示質心與編號、印出總數與各自面積;(d)進階：若物件相黏，加入距離轉換與分水嶺(第 9 章程式 9-4)正確分開，比較加入前後的計數差異;(e)量測校準：放一個已知尺寸的參考物(如硬幣)，換算出每像素代表的實際毫米數(呼應第 4 章相機校正)，輸出每個物件的實際尺寸。

Lab 10-3 ESP32-CAM 人臉偵測門禁

目標：實作叫獸開講與 10.6.3 節的端上人臉偵測。(a)先在電腦上以程式 10-5 跑通 Haar 級聯人臉偵測，調校 scaleFactor 與 minNeighbors，記錄「漏檢」與「誤報」如何隨參數變化;(b)改以 ESP32-CAM 的串流網址作為輸入來源(第 2、8 章);(c)當偵測到人臉持續數影格，觸發動作(存檔、亮 LED、印出時間戳記);(d)量測 FPS，比較「電腦端」與「ESP32-CAM 端上」的效能差異;(e)討論題：承第 2 章三架構——這個門禁若要加上「辨識是誰」的功能，偵測與辨識應各自放在端上、近端還是雲端?請畫出你的系統架構圖並說明理由。

Lab 10-4 型態學的代數性質實測

目標：把本章的幾條性質從「課本說」變成「我量過」。(a)寫一個 diff(a, b) 函式回傳兩張二值影像相異的像素數;(b)驗證對偶性：確認「對前景侵蝕」與「對背景膨脹再取反」的差為 0，斷開與閉合的對應關係也做一次;(c)驗證冪等性：把斷開重複 1 到 10 次，畫出「與做一次的差異」對次數的曲線，再用侵蝕做同一件事當對照組;(d)驗證結合律：比較「3×3 膨脹兩次」與「5×5 膨脹一次」，說明為什麼相等（提示：3×3 膨脹 3×3 會得到什麼）;(e)驗證可分離性：比較 k×k 方形結構元素與「1×k 接 k×1」的結果與耗時，把 k 由 3 掃到 31，畫出兩條時間曲線;(f)寫出 300 字的結論：這五條性質各自讓你在寫程式時少做了什麼決定。

Lab 10-5 順序之爭：自己掃一張表

目標：親手確認型態學運算不可交換，並找出自己場景的最佳順序。(a)取一張乾淨的二值影像當正確答案（可用幾何圖形合成），寫兩個函式分別加入「背景雜點」與「前景孔洞」，數量可調;(b)寫一個 err(m) 函式回傳與正確答案相異的像素數;(c)把雜點數與孔洞數各掃四到五檔，對每一種組合分別計算「先斷開再閉合」與「先閉合再斷開」的誤差，做成一張熱圖（參考圖 10-8）;(d)標出每一格的勝者，看看你的結論與本書的「只要有孔洞就先閉合」是否一致;(e)進階：加入交替序列濾波（結構元素由小到大，斷開閉合交替），看它能不能在整張表上都贏過固定順序，並記錄它多花了多少時間;(f)把你的熱圖與結論寫成一頁報告——這就是產線參數規格書的雛形。

Lab 10-6 從二值影像到量測報告

目標：把第 9、10 兩章串成一條能交差的完整管線。(a)拍一張散落的墊圈、螺帽或鈕扣照片（最好有背光，若無則請記錄光源條件）;(b)依序完成：灰階化 → CLAHE 或頂帽拉平背景（第 5 章、10.4 節）→ Otsu 或自適應門檻（第 9 章）→ 型態學清理（順序請用 Lab 10-5 的結論）→ 孔洞填補 → 邊界清除 → connectedComponentsWithStats;(c)用一枚已知外徑的標準件做標定，把像素換算成毫米，輸出每一件的外徑、面積與圓形度;(d)重複拍攝同一盤件十次（中間不要移動），計算每一項量測值的標準差——這是你的重複性;(e)刻意改變一個條件（移動光源、換曝光、把件稍微挪動），再拍十次，比較兩組的差異，判斷你的系統對什麼最敏感;(f)寫出一份含「量測值、重複性、已知的失效條件」三節的報告。請注意第三節最重要：一套說不出自己什麼時候會壞的系統，不能上線。

本章重點整理

- 二值影像每像素僅前景/背景兩值，多為第 9 章分割的產物;型態學以集合論為基礎，用結構元素探測前景形狀，結構元素的形狀與大小是處理關鍵。

- 侵蝕使前景縮水、消滅小細節(對灰階即最小值濾波);膨脹使前景長大、填補小孔(對灰階即最大值濾波);兩者為對偶運算。
- 斷開(先侵蝕後膨脹)去除背景雜點且維持主體大小;閉合(先膨脹後侵蝕)填補前景孔洞且維持主體大小;是分割後清理的常用組合。
- 型態學梯度求輪廓;頂帽萃取亮細節並校正不均勻背景(與同態濾波同功);黑帽萃取暗細節;Hit-or-Miss 偵測特定形狀，為細線化與骨架化的基礎。
- 連通元件標記回答「有幾個物件」並提供面積、外接矩形、質心等統計;距離轉換為每個前景像素計算到最近背景的距離，用於求骨架與分離相黏物件(分水嶺標記)。
- Viola-Jones 以 Haar 特徵、積分影像、AdaBoost 級聯三支柱達成即時人臉偵測，輕量而適合 ESP32-CAM 等端上部署;擅長正面人臉，側臉與遮擋則需第 16 章的深度學習方法。

習題

1. 二值影像通常從何而來?為什麼把影像化約成非黑即白反而有價值?分割產生的二值影像常有哪些缺陷?

   > *提示與參考解答：多來自第 9 章的分割(門檻、Otsu、邊緣偵測)。價值在於：當只關心物件的位置、形狀、數量時，二值化讓形狀分析極為高效。常見缺陷：前景內有小孔洞(反光/雜訊)、背景有孤立小白點、該分開的物件相黏、該相連的物件斷裂——這些正是型態學要清理的。*

2. 什麼是結構元素?它與空間濾波的核有何異同?為什麼選擇不同形狀的結構元素會影響結果?

   > *提示與參考解答：結構元素是型態學用來探測前景的小形狀，相當於濾波的核，但作用是集合運算而非數值加權。形狀(方/十字/圓/線)會影響結果：例如水平線形結構元素會強化水平結構、削弱垂直結構;圓形則各方向一視同仁。選對形狀與大小是型態學的關鍵。*

3. 說明侵蝕與膨脹的規則與效果，並指出它們與第 7 章哪兩種濾波等價。

   提示與參考解答：侵蝕：結構元素完全落在前景內時原點才保留，效果是前景縮水、消滅小細節，等價於灰階的最小值濾波(去鹽雜訊)。膨脹：結構元素與前景有交集即設為前景，效果是前景長大、填補小孔，等價於最大值濾波(去胡椒雜訊)。

4. 斷開與閉合各是什麼順序的組合?為什麼它們能在去除缺陷的同時大致維持物件原本大小?各自的口訣是什麼?

   提示與參考解答：斷開=先侵蝕後膨脹，去除背景雜點(「開」掉背景雜訊);閉合=先膨脹後侵蝕，填補前景孔洞(「閉」合前景破洞)。維持大小的原因：一縮一脹(或一脹一縮)相互抵消，主體回到接近原始尺寸，只有小於結構元素的缺陷被永久去除或填補。

5. 頂帽轉換如何校正不均勻的背景照明?它與第 6 章的哪個技術解決同一問題?兩者作用的域有何不同?

   提示與參考解答：頂帽=原圖減去斷開結果。用大結構元素做斷開會得到緩慢變化的背景照明，原圖減去它即抽出均勻照明下的前景亮細節。它與第 6.3.3 節的同態濾波解決同一問題(不均勻照明)：頂帽在空間域用形狀運算，同態濾波在頻率域用對數加高低頻分離。

6. 細線化與骨架化的目的是什麼?各舉一個應用。骨架化與距離轉換有何關聯?

   提示與參考解答：目的是把物件化約成單像素寬的線或中軸，以分析其拓撲結構。細線化應用：指紋脊線細化、文字筆畫分析。骨架化應用：電路走線分析、道路網路萃取。關聯：距離轉換的山脊(局部最大值連線)正是物件中軸，提供求骨架的另一途徑。

7. 連通元件標記回答什麼問題?它建立在第 3 章的什麼概念上?connectedComponentsWithStats 除了標記還提供哪些資訊?

   提示與參考解答：回答「畫面中有幾個相連的前景區域(物件)」。建立在第 3 章的連通性(4-連通或 8-連通，選擇不同計數可能不同)。額外資訊：每個元件的面積、外接矩形(位置與大小)、質心座標，可直接用於計數與量測，並以面積門檻濾除雜點。

8. 距離轉換如何幫助分開相黏的物件?請說明它在第 9 章分水嶺流程中扮演的角色。

> *提示與參考解答：對相黏的圓形物件做距離轉換，每個物件的中心離背景最遠、形成一個局部極大亮點。取這些亮點作為分水嶺的「確定前景」標記，由此注水，即可在兩物件的接縫處築起分界，正確分開它們。這就是第 9 章程式 9-4 中距離轉換那一步的原理。*

9. Viola-Jones 的三大技術支柱是什麼?各自解決了什麼問題?

> *提示與參考解答：一、Haar-like 特徵：以相鄰矩形亮度差捕捉人臉明暗規律(如眼比頰暗)。二、積分影像：預先累加，使任意矩形區域總和查表三次加減即得、與大小無關，解決海量特徵的計算速度。三、AdaBoost 級聯：從十八萬個特徵篩選少數有效者，並串成級聯前段快速淘汰非人臉、後段精細確認，達成即時偵測。*

10. 積分影像的「預先計算換取常數時間查詢」思想，與本章哪個技術相通?這屬於哪一類演算法設計思維?

> *提示與參考解答：與連通元件標記的「單次掃描完成統計」相通，也與部分實作中的累加思想一致。這屬於「空間換時間」的演算法設計思維：預先付出計算與儲存成本，換取後續查詢時的極高速度。*

11. detectMultiScale 的 scaleFactor 與 minNeighbors 各控制什麼?調大或調小如何影響「漏檢」與「誤報」?這與第 9、16 章的哪些取捨同類?

> *提示與參考解答：scaleFactor 是每次尺度縮放比例：愈接近 1 搜尋愈細密、愈不漏檢但愈慢。minNeighbors 是確認一個偵測所需的鄰近偵測數：愈大愈能抑制誤報但可能漏掉真臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 雙門檻、第 16 章物件偵測的信心門檻同類。*

12. (綜合題)為什麼在 ESP32-CAM 上偵測人臉，2001 年的 Viola-Jones 可能比 2020 年代的 YOLO 更合適?請以第 2 章三架構與第 9 章「方法沒有絕對優劣」的觀點論證。

> *提示與參考解答：ESP32-CAM 算力與記憶體極為有限，YOLO 等深度模型難以即時執行;Viola-Jones 輕量、不需 GPU，適合端上運算(低延遲、保護隱私、不需連網)。以第 2 章三架構看，輕量偵測放端上最務實;若需辨識身分再上傳雲端。呼應第 9 章：方法沒有絕對優劣，只有適不適合任務與硬體——在低功耗節點偵測正面人臉的場景，舊演算法反而勝出。*

13. 為什麼說「對灰階影像而言，侵蝕就是最小值濾波」不是巧合而是必然?請由侵蝕的集合定義推導出來。

    提示與參考解答：二值侵蝕要求「結構元素覆蓋範圍內每一點都是前景」，也就是「覆蓋範圍內的值最小者仍然是 1」。把這句話裡的 1 換成任意灰階，就得到「輸出 = 覆蓋範圍內輸入的最小值」，正是最小值濾波。膨脹的推導對稱：把「完全包含」換成「有交集」、「最小值」換成「最大值」。圖 10-4 把 cv2.erode 與 scipy 的 minimum_filter 逐像素比對，最大差異為 0。

14. 一張影像只有鹽雜訊（白點）。只做侵蝕會發生什麼事?請以 RMSE 說明，並指出正確的作法。

    提示與參考解答：侵蝕確實會消滅白點（取最小值），但它同時把每個像素都換成鄰域的最小值，整張影像跟著變暗。圖 10-4 實測：RMSE 由 20.9 不降反升。正確作法是做完整的斷開（蝕完再膨脹回來），RMSE 降到 10.1。更值得注意的是：對一張完全乾淨的影像做同樣的斷開，RMSE 已經是 10.1——兩者幾乎相等，表示雜訊已經被清光，剩下的全是運算本身的平滑代價。這個「先量上限再評估方法」的作法與圖 9-6 是同一套思路。

15. 請寫出侵蝕與膨脹的對偶關係，並說明它在實務上怎麼幫你省力。

    提示與參考解答：「對 A 侵蝕」等於「對 A 的補集膨脹之後再取補集」（結構元素需翻轉 180 度，但對稱的結構元素可省略）;斷開與閉合之間也有同樣的對偶關係。圖 10-5 把兩條式子逐像素驗證，相異的像素數都是 0。省力之處有二：一是只需要熟悉其中一半，另一半用「換個角度看背景」即可推得;二是當你猶豫該侵蝕還是膨脹時，可以改問「我是想縮前景，還是想縮背景」。

16. 膨脹滿足結合律，這對 cv2.erode/dilate 的 iterations 參數有什麼含意?它與第 5 章的可分離濾波有什麼關聯?

    提示與參考解答：結合律表示「用 B1 膨脹再用 B2 膨脹」等於「用 B1 膨脹 B2 所得的結構元素膨脹一次」，所以用 3×3 膨脹兩次等於用 5×5 膨脹一次——iterations 與放大結構元素是等效的。關聯：k×k 的方形結構元素可以拆成 1×k 的水平接 k×1 的垂直，運算量由 k^2 降到 $2k$，這正是第 5.3.3 節可分離濾波在型態學上的對應物。

17. 什麼是冪等性?斷開與閉合有，侵蝕沒有。這個差別對參數調校有什麼直接的影響?

    提示與參考解答：冪等性指「做一次與做無數次結果相同」。斷開的結果可以刻畫成「所有能完整塞進前景的結構元素的聯集」，既然輸出已經是這些結構元素的聯集，再斷開一次當然不會改變。圖 10-9 實測：斷開與閉合重複 2 到 5 次，與做一次相異的像素數都是 0;侵蝕重複 5 次則相差一萬四千多個像素。影響：斷開與閉合的 iterations 設 1 即可，要加重力道請放大結構元素;若你發現「多做幾次有差」，那表示你呼叫的其實不是斷開或閉合。

18. 一張二值影像同時有背景雜點與前景孔洞。「先斷開再閉合」與「先閉合再斷開」哪一個比較好?為什麼?

    提示與參考解答：型態學運算不可交換，兩者是不同的運算。圖 10-8 把兩種缺陷的數量各掃四檔、共 16 種組合實測：「先閉合再斷開」在 11 格勝出，「先斷開再閉合」只在 4 格勝出，而那 4 格裡有 3 格是「完全沒有孔洞」的情形。原因是斷開具有遞減性且不可逆——若前景布滿孔洞，先斷開會把物件咬成碎片，之後的閉合補不回來;先閉合則把物件補成實心，再斷開就能乾淨地掃掉背景雜點。原則：哪種缺陷佔優勢就讓對症的運算排在前面，只要前景有像樣的孔洞就先閉合。

19. 承第 5 題的頂帽轉換：為什麼它的結構元素必須比瑕疵大很多?做完頂帽之後，為什麼還是不能直接套 Otsu?

    提示與參考解答：用一個比瑕疵大得多的結構元素做斷開，小東西全被抹掉，剩下的就是緩慢變化的背景照明;原圖減去它，背景就被扣平。結構元素若比瑕疵小，瑕疵本身也會在斷開時存活下來，相減之後就消失了。圖 10-11 的實測回答了第二問：該圖的瑕疵只佔全圖 0.53% 的像素，遠低於圖 9-5 量到的「Otsu 需要前景至少約 3%」那條線，所以在頂帽結果上再做 Otsu 仍然失敗（IoU 0.014）。改用固定門檻 18 則得到 IoU 0.965，而且 17 到 24 之間都好用——因為頂帽已把背景壓到 0 附近，門檻變得非常好選。分工是：型態學負責拉平背景，門檻交給固定值或樣品標定。

20. 型態學梯度與第 9 章的 Sobel、Canny 有什麼不同?什麼叫內梯度與外梯度，各該在什麼時候用?

    提示與參考解答：型態學梯度 = 膨脹減侵蝕，基於形狀的集合運算而非微分。圖

10-10 實測：3×3 結構元素下型態學梯度的輪廓平均寬約 2.0 像素（向內外各長一格），內梯度（原圖減侵蝕）與外梯度（膨脹減原圖）各為 1.0，Canny 為 1.1。內梯度取得的是物件這一側的邊界，外梯度取得的是外側——這個「想要哪一側」的選擇權是微分型邊緣偵測給不了的。選用原則：已經二值化的影像用型態學梯度（一行寫完，且結構元素的形狀還能挑方向），灰階影像用微分法（定位較準）。

21. 計數之前為什麼要做孔洞填補與邊界清除?兩者的先後順序可以交換嗎?

    提示與參考解答：孔洞填補把物件內部封閉的背景補成前景，否則像墊圈這種環形物件量到的面積會少掉孔的部分——圖 10-13 實測為 4,624 對 5,525 像素，差了 16%，若驗收標準是面積，這個誤差足以讓良品被判成不良。邊界清除丟掉所有碰到影像邊緣的物件，因為它們被畫面切掉了一部分，面積與形狀都不可信;該圖做完之後元件數由 6 降到 4，與正解一致。順序不能交換：必須先填補再清除。反過來做，碰邊的物件會先被清掉，它內部的孔洞反而留下來變成假物件。

22. 4-連通與 8-連通會造成多大的差別?為什麼前景與背景應該用不同的連通性?另外，cv2.connectedComponents(img, 4) 這樣寫對嗎?

    提示與參考解答：差別取決於前景的形狀。圖 10-14 實測：硬幣的實心區塊兩種連通性都是 34 個，完全沒差;但同一張影像的 Canny 邊緣（細線結構）4-連通數出 1,093 個、8-連通只有 120 個，差了九倍——斜著走的邊緣在 4-連通下整條被切成碎片。前景與背景必須用不同的連通性，否則會違反 Jordan 曲線定理，出現「封閉曲線沒有把平面分成內外兩塊」的矛盾。最後一問：寫錯了。OpenCV 的簽章是 connectedComponents(image, labels, connectivity)，所以 4 被當成 labels 傳進去，連通性仍是預設的 8，而且不會報錯。必須寫 connectivity=4。

23. 連通元件標記為什麼只要掃描兩遍?它用到什麼資料結構?為什麼面積、外接矩形、質心是「免費」的?

    提示與參考解答：第一遍由上到下、由左到右，只看已掃過的鄰居，分三種情況給標籤（全新／沿用／合併並記錄等價）;第二遍把暫時標籤換成等價類的代表。等價關係用聯集-尋找（Union-Find）維護，加上路徑壓縮與按秩合併後，m 次操作的成本是 O(m α(n))，α 是反阿克曼函數，實務上小於 5，可視為常數，所以整體複雜度是 O(N)。統計量之所以免費，是因為面積是計數、外接矩形是四個極值、質心是座標累加和除以面積，三者都能在第一遍邊走邊累加，合併等價類時把兩份統計相加即可，不需要再掃一次影像。

24. 積分影像為什麼能加速 Haar-like 特徵的計算?加速比與矩形大小有什麼關係?級聯結構又省下了多少運算?

    提示與參考解答：積分影像的每一格存「從左上角到該處的所有像素總和」，建表只要掃一遍;之後任意矩形的總和都只要查四個角、做三次加減，成本與矩形大小無關。圖 10-17 實測：邊長 4 時快約 2 倍，邊長 256 時快約 45 倍——矩形愈大賺愈多，因為逐格相加的成本隨面積成長而查表是常數。級聯方面，圖 10-18 實算：一張 640×480 的畫面以 24×24 起始視窗、scaleFactor 1.1 掃描，共有 271,414 個候選視窗;Viola-Jones 的完整分類器有 6,061 個特徵，若每個視窗都跑完是 16.5 億次評估。論文報告平均每視窗只評估約 10 個特徵，總量降到 271 萬次，差了約 600 倍。

25. (綜合設計題)請為一條「墊圈外觀檢測與計數」的產線設計完整的二值影像處理管線，列出每一步的技術、參數決定方式與驗收指標，並說明若換成 OpenCV 5 需要注意什麼。

    提示與參考解答：一、成像：背光讓墊圈成為高對比剪影，關閉自動曝光（第 8 章），固定工作距離以便像素與毫米的標定（第 4 章）。二、二值化：背光下直方圖乾淨雙峰，用 Otsu;若仍有照明梯度，改用頂帽拉平背景再套固定門檻（圖 10-11）。三、清理：先看缺陷以哪種為主——墊圈內孔容易因反光產生破洞，所以通常先閉合再斷開（圖 10-8）;iterations 設 1，力道用結構元素大小控制（圖 10-9）。四、計數前處理：先孔洞填補（讓面積正確）再邊界清除（丟掉被畫面切到的件），順序不可反（圖 10-13）。五、量測：connectedComponentsWithStats 取面積、外接矩形、質心，並以 connectivity=8 明寫;內孔直徑可由「填補前後的面積差」反推，或用距離轉換取最大值（需用 DIST_MASK_PRECISE，圖 10-16）。六、驗收：以已知尺寸的標準件標定像素與毫米的換算，記錄面積、圓形度、內外徑的統計分布，要求跨班別重複性優於公差的三分之一;一旦漂移先檢查光源與鏡頭，而不是調參數。七、版本：若環境是 OpenCV 5，本章的型態學與連通元件 API 全部不受影響，但若管線中有用到 Haar 級聯（例如同時做人臉或特定圖樣偵測），該 API 已被移除，需改裝 OpenCV 4.x 或改用其他偵測器（10.6.4 節）。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 9(型態學影像處理之標準參考)。
- Viola, P., & Jones, M., Rapid Object Detection using a Boosted Cascade of Simple Features, CVPR, 2001.(本章叫獸開講之史料來源)
- Viola, P., & Jones, M., Robust Real-Time Face Detection, IJCV, 57(2), 2004.
- Serra, J., Image Analysis and Mathematical Morphology, Academic Press, 1982.(數學型態學經典)
- Soille, P., Morphological Image Analysis: Principles and Applications, 2nd ed., Springer, 2003.
- OpenCV 官方教學：Morphological Transformations 與 Cascade Classifier。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)